

TECHNISCHE UNIVERSITÄT
CHEMNITZ

Privacy-Preserving Source Code Vulnerability Detection and Repair using Retrieval-Augmented LLMs for Visual Studio Code

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc. Web Engineering

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Md Hafizur Rahman
Student ID: 810641
Date: 14.05.2026

Internal Supervisor: Abubaker Gaber M.Sc.
Internal Examiner: Dr.-Ing. Sebastian Heil

Abstract

Developers want security feedback directly in the IDE while they write code. Traditional static analysis tools work well for many rule-based problems, but they often miss issues that need deeper reasoning or security context. Large Language Models (LLMs) may help with vulnerability detection and repair, but they also bring problems such as unstable quality, many false positives, and privacy risks when source code is sent to cloud services.

This thesis studies a privacy-preserving way to detect and repair vulnerabilities by combining locally running LLMs with Retrieval-Augmented Generation (RAG) in a VS Code extension. The system runs on the developer’s device and uses locally stored CWE, CVE, and OWASP knowledge, so source code does not need to leave the machine. It provides two modes: a low-latency inline mode for IDE-triggered feedback, and an audit mode that adds retrieved security knowledge to the model’s analysis.

The system is evaluated on 101 JavaScript/TypeScript cases (71 vulnerable and 30 secure) based on CWE/OWASP-related concepts. The evaluation uses precision, recall, F1-score, false-positive rate, and latency. The results show trade-offs between speed, quality, and noise that depend on the model and prompting mode. The best F1-score (65.31%) is reached by `qwen3:8b` with RAG, with a false-positive rate of 20.00%. Overall, the study shows that local privacy-preserving LLM workflows can support useful vulnerability analysis, but careful model choice and false-positive control are still necessary for practical use.

Keywords: source code security, vulnerability detection, secure code repair, LLMs, RAG, privacy-preserving systems, VS Code

Acknowledgment

I am grateful to everyone who supported me during this Master's thesis.

My sincere thanks go to my internal supervisor, *Abubaker Gaber, M.Sc.*, for his continuous guidance, constructive feedback, and thoughtful discussions throughout the project. His support was essential in shaping both the technical direction and the academic quality of this work.

I also thank the academic staff of the Master's program for providing a strong foundation and a stimulating learning environment.

I am equally thankful to my colleagues and peers for their encouragement and for creating a supportive atmosphere during the study period.

Most of all, I thank my family for their patience, trust, and constant encouragement. Their support made it possible for me to complete this thesis.

Task Description

Traditional rule-based security analysis tools are effective for detecting predefined vulnerability patterns but can struggle with context-dependent weaknesses that require semantic reasoning. Cloud-based language model assistants provide stronger contextual analysis, yet compromise confidentiality and reproducibility because proprietary source code must leave the local environment. This thesis designs and evaluates a fully local, privacy-preserving secure coding assistant for Visual Studio Code that integrates large language models (LLMs) with Retrieval-Augmented Generation (RAG). The prototype is built around a no-source-code-exfiltration boundary: analyzed code and IDE-derived context are processed on-device and sent only to a local LLM runtime.

The assistant combines two complementary modes: real-time inline diagnostics that surface debounced vulnerability alerts during active development, and asynchronous audit and question-answering modes for deeper inspection. In audit mode, the prompt can optionally be grounded with a local retrieval index over security guidance (e.g., CWE/OWASP-aligned summaries and optionally cached public CVE/NVD metadata). In question-answering mode, users provide explicit file/folder context and receive narrative security review guidance. A modular architecture separates the LLM inference layer, the retrieval pipeline, and the Visual Studio Code extension interface. Privacy is enforced primarily through local deployment (no external inference APIs) and by limiting knowledge refresh to public metadata; rerunability is supported by version-pinned artifacts where available and by explicit reporting of model fingerprints and runtime settings. The threat model includes prompt injection via attacker-controlled code text and retrieval-quality risks; the prototype mitigates these risks primarily through strict JSON output contracts in diagnostic workflows, defensive parsing, and curated retrieval sources, while stronger provenance controls remain future work.

Evaluation is conducted on a project-authored JavaScript/TypeScript suite aligned to CWE/OWASP concepts, combining vulnerable and secure/negative cases to quantify both detection quality and false-positive behavior. The comparison covers LLM-only and LLM+RAG prompting, and includes baseline SAST tools (Semgrep, CodeQL, and ESLint security rules) to contextualize trade-offs.

Evaluation metrics include precision, recall, F1-score, secure-sample false-positive rate, JSON parse success, and latency. Privacy and reproducibility are assessed primarily through architectural/configuration evidence (local inference boundary, optional refresh behavior) and run-provenance reporting under controlled local

execution.

Contents

Abstract	ii
Acknowledgment	iii
Task Description	iv
Contents	vi
List of Figures	xii
List of Tables	xiv
List of Abbreviations	xvi
1. Introduction	1
1.1. Problem Description	1
1.2. Motivating Scenario	2
1.3. Scope of the Thesis	3
1.4. Objectives	5
1.5. Thesis Outline	5
2. Analysis	7
2.1. Requirements Analysis	7
2.1.1. R1: Accuracy	8
2.1.2. R2: Consistency	9
2.1.3. R3: Repair Quality	10
2.1.4. R4: Usability	11

CONTENTS

2.1.5. R5: Privacy	12
2.2. Related Work	13
2.2.1. Traditional Static Application Security Testing	13
2.2.2. LLM-Based Vulnerability Detection and Repair	15
2.2.3. Retrieval-Augmented Generation for Secure Coding	17
2.2.4. Privacy-Preserving and IDE-Integrated Approaches	19
2.2.5. Positioning of This Thesis	21
2.2.6. Critical Comparison Across Paradigms	24
2.3. Summary	25
3. Concept	26
3.1. Concept Derivation from Analysis Results	26
3.1.1. From Cloud-Based to Privacy-Preserving Local Deployment	27
3.1.2. Retrieval-Augmented Generation for Security Knowledge Grounding	27
3.1.3. IDE Integration for In-Context Security Assistance	28
3.1.4. Modular Component Architecture	29
3.1.5. Context-Aware Analysis with Code Understanding	29
3.1.6. Explainability Through Structured Reasoning	30
3.1.7. Deterministic Inference and Consensus-Based Detection	31
3.1.8. Two-Stage Detection and Repair Pipeline	31
3.2. System Components	32
3.2.1. Context Extraction Component	32
3.2.2. Knowledge Retrieval Component (RAG)	33
3.2.3. Vulnerability Detection Component	34
3.2.4. Repair Generation Component	34
3.2.5. IDE Integration Layer	35
3.2.6. Supporting Subsystems	35
3.3. Detection Workflows	36
3.3.1. Real-Time Function-Level Diagnostics	36

CONTENTS

3.3.2.	On-Demand File Diagnostics	37
3.3.3.	Interactive Analysis and Follow-up Q&A	37
3.3.4.	Workspace Scan and Security Dashboard	37
3.3.5.	Repair Suggestion Workflow	38
3.3.6.	Workflow Selection in Practice	38
3.4.	System Architecture and Design	38
3.4.1.	Context View: System Boundary and External Interactions . .	39
3.4.2.	Container View: Internal Component Structure	40
3.4.3.	Process View: End-to-End Workflows	43
3.4.4.	Sequence Diagrams: Concrete Detection Flows	46
3.5.	Summary	52
4.	Implementation	53
4.1.	Technology Stack and Rationale	53
4.2.	System Workflow	55
4.3.	Core Component Implementation	58
4.3.1.	Context Extraction and Scoping	58
4.3.2.	LLM Analyzer (Local JSON-Only Output)	58
4.3.3.	Diagnostics Mapping and Localization	60
4.3.4.	RAG Manager (Local Retrieval)	60
4.3.5.	Vulnerability Knowledge Updates	60
4.3.6.	Analysis Cache	61
4.3.7.	Workspace Scanner and Dashboard	62
4.3.8.	Privacy Boundary and Threat Model Considerations	63
4.4.	User Interface	63
4.4.1.	Inline Diagnostics and Hover Explanations	63
4.4.2.	Quick Fixes for Repair Suggestions	64
4.4.3.	Interactive Analysis View and Contextual Q&A	64
4.4.4.	Workspace Security Dashboard	64
4.4.5.	Model and RAG Controls	65

CONTENTS

4.4.6.	Human Factors and Safe Adoption	65
4.5.	Repair Safety and Limitations	66
4.5.1.	Why Automated Repair Validation Was Deprioritized	66
4.5.2.	Current Repair Safety Mechanisms	67
4.5.3.	Observed Repair Patterns and Quality	68
4.5.4.	Developer Workflow for Repair Application	69
4.6.	Implementation Summary	70
5.	Evaluation	71
5.1.	Datasets	71
5.2.	Evaluation Metrics	73
5.3.	Experimental Setup	75
5.3.1.	Data Merging and Run-to-Run Variability	78
5.4.	R1: Accuracy	79
5.4.1.	Detection Quality Across Configurations	79
5.4.2.	RAG Ablation Impact on Accuracy	81
5.4.3.	Per-Category Detection Breakdown	81
5.4.4.	SAST Baseline Comparison	82
5.4.5.	Level Mapping	82
5.5.	R2: Consistency	83
5.5.1.	Structured Output Stability	83
5.5.2.	Inter-Run Detection Agreement	83
5.5.3.	Label and Severity Stability	84
5.5.4.	Consistency by Vulnerability Category	85
5.5.5.	SAST Baseline Consistency	85
5.5.6.	Level Mapping	86
5.6.	R3: Repair Quality	86
5.6.1.	Repair Generation Rate and Correctness	86
5.6.2.	Repair Quality by Vulnerability Type	87
5.6.3.	Representative Examples	88

CONTENTS

5.6.4. Level Mapping	88
5.7. R4: Usability	88
5.7.1. Latency Across Configurations	89
5.7.2. Latency Component Breakdown	90
5.7.3. Latency Feasibility for IDE Workflows	90
5.7.4. Level Mapping	90
5.8. R5: Privacy	91
5.8.1. Verification Results	91
5.8.2. Evidence	92
5.8.3. Comparison with Cloud-Based Tools	92
5.8.4. Level Mapping	93
5.9. Summary	93
5.9.1. Requirement Compliance	93
5.9.2. Key Findings	94
5.9.3. Limitations	95
5.9.4. Threats to Validity	95
6. Conclusion	96
6.1. Summary of Contributions	96
6.2. Key Findings	96
6.3. Answers to Research Questions	97
6.4. Implications for Practice	97
6.5. Limitations	98
6.6. Responsible Use	98
6.7. Future Work	99
6.8. Conclusion	100
A. Privacy Verification Evidence	101
A.1. Network Traffic Analysis	101
A.1.1. Test Configuration	101

CONTENTS

A.1.2. Monitoring Method	101
A.1.3. Results	101
A.1.4. Configuration Validation	102
A.2. Privacy Boundary Architecture	102
A.2.1. Trust Boundary Definitions	103
A.2.2. Out-of-Scope Threats	103
A.3. Offline Operation Mode	103
A.4. Privacy Compliance Summary	104
Bibliography	105
Declaration of Originality	111

List of Figures

3.1.	C4 Context diagram showing Code Guardian within its operational environment. The system communicates with VS Code (IDE platform), Ollama (local LLM on localhost:11434), and a local vector database, all within the privacy boundary (green dashed). CVE/NVD APIs (gold, dashed) are the only optional external connection and do not receive source code.	40
3.2.	C4 Container diagram showing internal components. The VS Code Extension handles triggers, caching, and UI rendering. The Local Analysis Backend performs LLM inference with JSON-mode and multi-pass consensus filtering, RAG orchestration, and Stage 2 repair generation. Vector Database and Result Cache are local data stores. All source code stays within the system boundary.	42
3.3.	Process view showing two primary workflows with swim lanes. Workflow 1: Real-time function analysis with debouncing (800 ms) and caching, where cache hits bypass LLM inference and cache misses invoke local analysis. Workflow 2: On-demand analysis with optional RAG; when enabled, the backend generates embeddings, performs vector search (runtime default $k=3$; evaluation uses $k=5$), and augments the prompt before LLM invocation. When RAG is disabled, analysis proceeds directly to LLM inference without retrieval. Color coding: user events (yellow), extension (orange), backend (purple), LLM (green). Privacy boundary (green dashed): all processes execute on localhost.	45
3.4.	Sequence diagram: Inline mode with cache hit. No LLM invocation is required for previously analyzed code.	47
3.5.	Sequence diagram: Audit mode with RAG. The backend retrieves security knowledge via vector search, runs two detection passes with consensus filtering, and generates repairs for confirmed findings. . . .	49
3.6.	Sequence diagram: Real-time detection flow with debouncing. An 800 ms timer prevents excessive LLM invocations during active typing. . . .	51

LIST OF FIGURES

5.1.	F1 scores across all configurations. LLM-based approaches outperform SAST baselines on recall-driven F1, with qwen3:8b+RAG achieving the highest score.	80
5.2.	RAG ablation: F1 comparison by model. RAG improves qwen3 models but degrades gemma3 and code llama	81
5.3.	Recall by vulnerability category for qwen3:8b+RAG . Injection-style vulnerabilities (SQL, XSS, command) are detected most reliably. Niche categories (NoSQL, deserialization) remain challenging.	82
5.4.	JSON parse success rate by model and mode. Larger models produce more reliably structured output. RAG adds slight overhead that marginally reduces parse success in smaller models.	83
5.5.	Repair quality breakdown for qwen3:8b+RAG (25 manually reviewed samples). Most repairs are semantically correct and strategy-aligned, but less than half contain directly executable code.	87
5.6.	Median and p95 latency across configurations. SAST baselines are orders of magnitude faster. Among LLM configurations, smaller models trade accuracy for speed.	89
A.1.	Local privacy boundary for code analysis workflows.	102

List of Tables

1.1. Threat model summary for Code Guardian.	4
2.1. Requirements overview for Code Guardian.	7
2.2. Evaluation scale for R1: Detection Accuracy (example thresholds). . .	9
2.3. Evaluation scale for R2: Detection Consistency (example thresholds). .	10
2.4. Evaluation scale for R3: Repair Quality (example thresholds).	11
2.5. Evaluation scale for R4: Usability (example thresholds).	12
2.6. Verification checklist for R5: Privacy.	13
2.7. Evaluation of traditional SAST tools against requirements R1–R5. . .	14
2.8. Evaluation of LLM-based vulnerability detection approaches against requirements R1–R5.	17
2.9. Evaluation of RAG-based secure coding approaches against require- ments R1–R5.	19
2.10. Evaluation of privacy-preserving and IDE-integrated approaches against requirements R1–R5.	20
2.11. Positioning comparison with representative security assistance ap- proaches.	22
2.12. Unified requirement comparison of all reviewed approaches and Code Guardian.	23
2.13. Critical comparison of major security-assistance paradigms.	25
3.1. Comparison of detection flow characteristics.	52
5.1. Run configuration used for reported results.	78
5.2. Detection accuracy across all configurations.	80
5.3. Inter-run detection agreement (3 runs per sample, 71 vulnerable samples). .	84

LIST OF TABLES

5.4. Label and severity stability among consistently detected samples (3/3 runs).	85
5.5. Repair generation rate by vulnerability type (qwen3:8b+RAG).	87
5.6. End-to-end latency per analysis request (milliseconds).	89
5.7. Latency feasibility for IDE workflow modes.	90
5.8. Privacy verification results for R5.	91
5.9. Privacy comparison: Code Guardian vs. cloud-based security tools. .	92
5.10. Requirement compliance summary for Code Guardian (qwen3:8b+RAG). .	94
6.1. Recommended deployment profiles from thesis results.	98
A.1. Privacy-preserving configuration parameters.	102
A.2. Trust boundaries and data residency guarantees.	103
A.3. Privacy requirement compliance evidence summary.	104

List of Abbreviations

AST	Abstract Syntax Tree	LLM	Large Language Model
CVE	Common Vulnerabilities and Exposures	LRU	Least Recently Used
CWE	Common Weakness Enumeration	OWASP	Open Worldwide Application Security Project
F1	F1 Score	RAG	Retrieval-Augmented Generation
FPR	False Positive Rate	SAST	Static Application Security Testing
HNSW	Hierarchical Navigable Small World	VS Code	Visual Studio Code
IDE	Integrated Development Environment		

1. Introduction

Modern software systems underpin critical infrastructure, enterprise platforms, and consumer services. Vulnerabilities introduced during implementation remain a persistent attack vector, with injection flaws, cross-site scripting, and insecure data handling among the most frequently reported weakness classes [1, 2]. Developers are expected to write secure code, but the tools available to them either lack contextual reasoning or require transmitting source code to external services. This chapter introduces the problem, presents a motivating scenario, defines the scope and objectives, and outlines the structure of the thesis.

1.1. Problem Description

Static Application Security Testing (SAST) tools such as Semgrep and CodeQL have been the default approach to automated vulnerability detection for over a decade. They are deterministic, fast, and easy to integrate into CI/CD pipelines [3, 4]. These tools work well for known patterns in controlled settings, but they break down when vulnerabilities require reasoning about context, data flow across framework boundaries, or developer intent.

A major limitation is shallowness. Rule-based scanners match syntactic patterns but cannot explain *why* a finding matters in the surrounding code context. They do not distinguish a genuinely dangerous sink from one that is already guarded by upstream validation. As a result, developers frequently dismiss SAST warnings as noise, especially under delivery pressure [5, 6].

Another limitation is the absence of repair guidance. SAST tools flag problems but rarely suggest how to fix them. The developer must independently research the vulnerability class, identify the appropriate mitigation, and apply it correctly. This gap between detection and remediation slows the feedback loop and increases the chance that findings are ignored.

Large Language Models (LLMs) address both limitations: they can reason about code context, explain findings, and generate concrete repair suggestions [7, 8]. However, most LLM-powered security tools rely on cloud inference, which means the developer’s source code must leave the machine. For organizations with confidentiality requirements, regulatory constraints such as GDPR, or air-gapped development environments, this is not acceptable [9, 10].

1. Introduction

These problems persist because current approaches optimize for one dimension at the expense of others. SAST tools prioritize determinism and privacy but sacrifice contextual reasoning. Cloud-based LLM assistants prioritize reasoning quality but sacrifice privacy. No existing tool combines contextual vulnerability reasoning, actionable repair suggestions, and strict source-code privacy in an IDE-integrated workflow.

To overcome these limitations, a local and modular design is needed — one that keeps all inference on the developer’s machine, grounds model reasoning in curated security knowledge, and integrates findings directly into the editor with developer-controlled remediation.

1.2. Motivating Scenario

To illustrate these challenges in a realistic development setting, consider a typical workflow at a fintech company building a Node.js backend.

A developer is implementing an Express.js endpoint that retrieves user account details. The handler reads a user ID from the request query string and constructs a SQL query to look up the account. The code is straightforward, the tests pass, and the pull request is approved by a colleague:

Listing 1.1: Vulnerable Express.js endpoint with SQL injection

```
app.get('/api/account', (req, res) => {
  const userId = req.query.id;
  const query = "SELECT * FROM accounts WHERE id = '" + userId + "'";
  db.query(query, (err, results) => {
    res.json(results);
  });
});
```

The query is built by string concatenation. An attacker who sends `id=' OR '1'='1` retrieves every account in the database. No one notices the vulnerability until a penetration test three months later.

This scenario reveals four gaps in existing security tooling:

- **A linter stays silent.** ESLint with security plugins does not flag this pattern because the concatenation is syntactically valid JavaScript. The tool has no concept of tainted input or SQL semantics.
- **A SAST scanner gives a generic warning.** Semgrep might flag string concatenation in a query, but the warning says “potential SQL injection” without explaining whether the specific data flow from `req.query` to `db.query` is actually dangerous, or how to fix it.

- **A cloud-based LLM assistant explains and repairs — but requires sending the code offsite.** A tool like GitHub Copilot could identify the injection, explain the tainted data flow, and suggest replacing the concatenation with a parameterized query (`db.query("SELECT * FROM accounts WHERE id = ?", [userId])`). But this requires transmitting the source code to an external server.
- **No tool provides all three: context-aware detection, actionable repair, and privacy.** The developer needs an assistant that can reason about the vulnerability in context, suggest a concrete fix, and do so without the code leaving the machine.

Now imagine a local AI assistant integrated into VS Code. As the developer writes the endpoint, the assistant analyzes the enclosing function, identifies the unsanitized `req.query.id` flowing into a SQL string, and produces a finding:

SQL Injection (CWE-89): User-controlled input `req.query.id` is concatenated into a SQL query without sanitization. Replace with a parameterized query: `db.query("SELECT * FROM accounts WHERE id = ?", [userId])`.

The finding appears as an inline diagnostic in the editor. The developer hovers to read the explanation, clicks the quick-fix action to preview the parameterized query, and applies the patch — all without the code leaving the machine. This is what Code Guardian provides: contextual detection, grounded explanation, and developer-controlled repair, running entirely on localhost.

1.3. Scope of the Thesis

This thesis focuses on **JavaScript/TypeScript** projects in **Visual Studio Code**. The primary privacy goal is **no source-code exfiltration**: all code and IDE context are processed exclusively by local services running on the developer’s machine. The system is designed as a VS Code extension that connects to a local Ollama instance for LLM inference, with an optional retrieval-augmented generation (RAG) module backed by a local HNSW vector store for security knowledge grounding.

The technical scope includes local LLM inference via Ollama, function-level code scoping, structured JSON output enforcement, and retrieval-augmented prompting with curated security knowledge bases (CWE descriptions, OWASP guidance). No model fine-tuning or training is performed; the focus is on system-level orchestration — prompt design, output parsing, caching, and IDE integration — using off-the-shelf local models served through Ollama [11].

The evaluation is limited to the accuracy, consistency, and latency of the structured output produced from code snippets in a controlled test harness. This thesis does

not cover areas such as multi-language support beyond JavaScript/TypeScript, real-time collaborative editing, endpoint hardening, hardware-level attacks, or enterprise identity and network controls. Optional knowledge-base refresh may fetch *public* vulnerability metadata (for example CVE/CWE descriptions) and can be disabled for fully offline operation.

Research Questions

The thesis addresses three research questions:

- **RQ1 (Feasibility):** To what extent can a locally deployed LLM integrated into VS Code provide useful vulnerability detection and repair suggestions without transmitting source code to external services?
- **RQ2 (Grounding):** How does retrieval-augmented prompting influence detection quality and the qualitative grounding of explanations compared to an LLM-only configuration?
- **RQ3 (Practicality):** What performance mechanisms (scoping, caching, debouncing) are necessary to make LLM-based security feedback usable within interactive IDE workflows?

Threat Model and Trust Assumptions

The main threat addressed is unintended disclosure of proprietary code through external analysis services. Local inference reduces this risk surface but does not remove all security risks.

Table 1.1.: Threat model summary for Code Guardian.

Threat class	Assumption / attack path	Design response in this thesis
Source-code exfiltration	External API calls could expose proprietary code or derived context.	Local inference only; prompts and code remain on localhost in evaluated workflows.
Prompt injection	Attacker-controlled code/comments attempt to override analysis instructions [12].	Strict JSON contract, defensive parsing, and user approval before applying any repair.
Retrieval poisoning	Low-quality or malicious knowledge entries reduce grounding quality.	Curated local knowledge sources and explicit retrieval scope; stronger provenance controls left for future work.
Local host compromise	Malware or untrusted local users access local code, prompts, or caches.	Out of scope; assumes trusted host and OS-level hardening.

1.4. Objectives

The main objective of this thesis is to design and evaluate an IDE-integrated assistant — called Code Guardian — that provides contextual vulnerability detection and repair suggestions while preserving source-code confidentiality. The system aims to overcome the limitations of existing approaches by combining local LLM inference with retrieval-augmented generation in a privacy-preserving architecture that runs entirely on the developer’s machine.

To achieve this goal, the following three specific objectives are defined:

1. **Design a privacy-preserving VS Code extension for local vulnerability analysis** that separates the process into a two-stage pipeline: detection followed by developer-controlled repair suggestion. The extension enforces structured JSON output for machine-checkable diagnostics, integrates an optional RAG module backed by curated security knowledge (CWE descriptions, OWASP guidance), and implements performance mechanisms — function-level scoping, LRU caching, and debounced triggering — for practical IDE interaction latency.
2. **Evaluate system performance across local model configurations** using a documented evaluation harness with 101 test cases spanning 14 CWE categories. Five local models are tested in both LLM-only and LLM+RAG modes, reporting precision, recall, F1, false-positive rate, JSON parse success, and end-to-end latency. Results from all configurations are compared to identify the most effective trade-off between detection quality, output robustness, and response time.
3. **Assess the practical impact of retrieval-augmented prompting** by comparing LLM-only and LLM+RAG configurations to determine whether grounding model reasoning in external security knowledge improves detection quality and explanation grounding, or whether the added retrieval context introduces noise that degrades specific model configurations.

These objectives guide the development and validation of a system that is not only technically functional but also practical for interactive developer workflows. In this thesis run, `qwen3:8b` with RAG provides the strongest overall trade-off among tested configurations (F1 65.31%, FPR 20.00%, median latency ~2.7s; Chapter 5).

1.5. Thesis Outline

Following this introduction, the thesis begins with a review of related work on static analysis tools, LLM-based vulnerability detection, and retrieval-augmented generation for secure coding. The literature is analyzed to identify current limitations and to derive requirements for a system that combines contextual reasoning, repair

1. Introduction

generation, and source-code privacy. This provides the conceptual foundation for understanding the need for a tool like Code Guardian.

The next chapter introduces the overall system concept. It presents the architecture of Code Guardian together with key design decisions such as the two-stage detection-then-repair pipeline, structured JSON output enforcement, and the optional RAG module. After outlining the concept, the implementation chapter describes the VS Code extension, technology choices, and how core components — such as the analyzer, RAG manager, analysis cache, and diagnostic rendering — were developed, with an emphasis on modularity and local execution.

The evaluation chapter examines system performance across five local models in both LLM-only and LLM+RAG configurations, using metrics such as precision, recall, F1, false-positive rate, JSON parse success, and latency. The thesis concludes by summarizing key findings, discussing limitations, and outlining opportunities for future work.

2. Analysis

This chapter defines the requirements for automated vulnerability detection and repair assistance in an IDE and positions the thesis approach against related work. Following the style of the reference theses, the chapter first derives the requirements baseline, then reviews existing approaches, and finally summarizes the implications for the proposed concept.

2.1. Requirements Analysis

The thesis targets local IDE assistance for JavaScript/TypeScript development: detect vulnerabilities, explain them, and propose repairs without code exfiltration. Compared with rule-based SAST, the approach adds LLM reasoning and optional retrieval grounding, but must still satisfy strict operational constraints.

Based on the problem statement and related work, five requirements define the evaluation baseline. Table 2.1 summarizes them before they are discussed in detail.

Table 2.1.: Requirements overview for Code Guardian.

ID	Requirement	Operational intent
R1	Accuracy	Find real vulnerabilities with useful precision, recall, and controlled false positives, using context-aware reasoning.
R2	Consistency	Produce stable findings and structured output under fixed settings.
R3	Repair quality	Suggest concrete, security-improving fixes while keeping the developer in control.
R4	Usability	Deliver feedback at latencies that fit normal IDE workflows.
R5	Privacy	Keep all analysis local with no source-code transmission.

The following subsections define these requirements in detail and map them to measurable criteria.

2.1.1. R1: Accuracy

R1 addresses accuracy. Code Guardian should find real vulnerabilities while keeping false alarms low. In an IDE workflow, accuracy means not just finding issues, but producing results that developers can trust and act on. This includes context-aware reasoning: the system should tell apart code that is actually vulnerable from code that only looks suspicious, by considering surrounding logic like input validation and sanitization.

This requirement comes from a key trade-off in security tools. Traditional tools often produce too many false warnings because they match surface patterns without understanding context [5, 4]. LLM-based systems can both over-warn and miss real issues, depending on the model and prompt [13, 14]. Without enough context, they may give confident but wrong explanations. A useful tool must be judged by how correct its findings are, not just by whether it can explain them.

For this thesis, R1 means detection quality must be good enough to deserve developer attention under local deployment. This includes recognizing when mitigation is already present and avoiding false conclusions when context is missing. The design uses scoped code extraction, structured prompts, optional retrieval grounding, comparison with rule-based tools, and clear reporting of precision, recall, F1, and false-positive rate (FPR).

Accuracy has four key parts. Precision measures how many reported issues are real, since too many false alarms cause developers to ignore the tool. Recall measures how many real vulnerabilities the system finds, including those that depend on data flow and surrounding context. F1 balances both into a single score. The secure-sample FPR tracks how often the system flags safe code as vulnerable, which is the best predictor of whether developers will accept the tool [5].

To allow comparison with other tools and across Code Guardian’s own settings, we define a four-level scoring scale. The thresholds follow standard conventions from information retrieval and software engineering [5, 13]. Table 2.2 shows this scale.

Table 2.2.: Evaluation scale for R1: Detection Accuracy (example thresholds).

Visual Score	Interpretation (with thresholds)
	High accuracy. $F1 \geq 0.75$, precision ≥ 0.70 , recall ≥ 0.70 , and $FPR \leq 0.20$. The system finds vulnerabilities well with few false alarms. Suitable for always-on IDE use.
	Medium accuracy. $F1$ in $[0.60, 0.75)$, precision ≥ 0.55 , recall ≥ 0.55 , and FPR in $(0.20, 0.40]$. Results are worth reviewing with some manual checking needed. Suitable for on-demand or audit-mode use.
	Low accuracy. $F1$ in $[0.45, 0.60)$, or precision/recall below 0.55, or FPR in $(0.40, 0.70]$. Findings need heavy manual checking. Only useful alongside rule-based tools.
	Insufficient accuracy. $F1 < 0.45$ or $FPR > 0.70$. Detection is too weak or false alarms too frequent. The system is not useful in practice.

2.1.2. R2: Consistency

R2 addresses detection consistency. Code Guardian should produce stable results when the same code is analyzed multiple times with the same settings. In security analysis, unstable output quickly reduces trust and makes it harder for developers to act on findings.

This requirement matters because LLMs are nondeterministic. They can produce different outputs, schema errors, or changed labels across runs [13, 8]. Even if detection quality is good, inconsistent output makes a tool unsuitable for IDE use. Developers need to trust that warnings will not change randomly between runs [5].

Consistency has four key parts. First, structured output: the system must return valid JSON that follows the expected schema, since broken output is useless for IDE automation. Second, detection agreement: running the same code twice should produce the same findings. Third, labeling: vulnerability types and severity ratings should stay the same across runs. Fourth, localization: reported line ranges should not shift between runs.

For this thesis, R2 means stable JSON output, stable findings across repeated runs, and repeatable labels and locations. The design uses a strict JSON contract, defensive parsing, scoped prompts, caching, and optional retrieval grounding to reduce random variation.

2. Analysis

We define a four-level scale for R2. Output stability is measured by JSON parse success rate. Detection agreement is measured by inter-run agreement rate or Cohen’s kappa. Table 2.3 shows this scale.

Table 2.3.: Evaluation scale for R2: Detection Consistency (example thresholds).

Visual Score	Interpretation (with thresholds)
	High consistency. JSON parse rate $\geq 99\%$, detection agreement ≥ 0.90 , and label/severity $\kappa \geq 0.80$. Output is stable and reliable. Suitable for always-on IDE use.
	Medium consistency. JSON parse rate in $[95\%, 99\%)$, detection agreement in $[0.75, 0.90)$, or κ in $[0.60, 0.80)$. Mostly stable with occasional changes. Suitable for on-demand or audit-mode use.
	Low consistency. JSON parse rate in $[85\%, 95\%)$, detection agreement in $[0.60, 0.75)$, or κ in $[0.40, 0.60)$. Frequent variation. Findings need repeated confirmation before acting.
	No consistency. JSON parse rate $< 85\%$, detection agreement < 0.60 , or $\kappa < 0.40$. Output is unreliable. The system cannot produce repeatable findings.

2.1.3. R3: Repair Quality

R3 addresses repair quality. Code Guardian should not only detect vulnerabilities but also suggest concrete fixes. A good repair is specific to the affected code, follows recognized secure-coding practices, and is presented as a suggestion that the developer can review and accept or reject.

This matters because developers often receive warnings without usable guidance on how to fix them, which increases effort and reduces follow-through [5, 6]. At the same time, research on automated program repair shows that generated patches are only useful when they fix the root cause without introducing new problems [15].

For Code Guardian, R3 means repairs should target the affected code region, use established mitigations like parameterization, input validation, and safer API choices, and be delivered as optional diffs rather than automatic edits. The developer always keeps control over whether to apply a fix.

To measure repair quality, we define a three-level scale. The metrics focus on whether repairs are syntactically valid, address the reported vulnerability type, and follow recognized secure-coding patterns. Table 2.4 shows this scale.

Table 2.4.: Evaluation scale for R3: Repair Quality (example thresholds).

Visual Score	Interpretation (with thresholds)
	High. $\geq 75\%$ of repairs are syntactically valid, address the correct vulnerability type, and follow established secure-coding patterns. Fixes are specific and ready for developer review.
	Medium. $[50\%, 75\%)$ of repairs are valid and relevant. Most fixes target the right issue but may use generic patterns or need minor edits before applying.
	Low. $< 50\%$ of repairs are valid and relevant. Fixes are mostly unusable, too generic, or miss the root cause.

2.1.4. R4: Usability

R4 addresses usability. Security feedback is only useful if it arrives fast enough to fit normal editing workflows. A tool that takes too long will be ignored, no matter how accurate it is.

This is especially important for local LLM-based systems because model inference, retrieval, and prompt processing can make analysis noticeably slower than traditional diagnostics [16, 17]. Growing delay breaks the developer’s flow and reduces willingness to use the tool.

For Code Guardian, R4 means that both inline and on-demand analysis must stay practical on real hardware. The design uses function-level scoping, debouncing, caching, and separate modes for lightweight inline checks versus heavier full-file scans.

To measure usability, we define a three-level scale based on end-to-end latency. The thresholds reflect established response-time guidelines from HCI research [16, 17]. Table 2.5 shows this scale.

2. Analysis

Table 2.5.: Evaluation scale for R4: Usability (example thresholds).

Visual Score	Interpretation (with thresholds)
	High. Median latency $\leq 5s$ for inline analysis, $\leq 15s$ for full-file scans. Feedback feels responsive and fits normal editing flow.
	Medium. Median latency in $(5s, 15s]$ for inline, $(15s, 30s]$ for full-file. Noticeable delay but still practical for on-demand use.
	Low. Median latency $> 15s$ for inline or $> 30s$ for full-file. Too slow for regular IDE use. Only practical for planned audit scans.

2.1.5. R5: Privacy

R5 addresses privacy. Code Guardian must detect and repair vulnerabilities without sending source code to external services. All prompts, findings, retrieved context, and generated fixes should stay on the developer's machine.

This matters because source code often contains proprietary logic or regulated information. Many organizations cannot use cloud-based analysis tools for this reason. Partial redaction does not solve the problem, since security-relevant context is spread across the code and may be lost when stripped.

For Code Guardian, R5 means local model inference, local retrieval, and clear separation between code-bearing workflows and optional public-metadata refresh paths. The design uses Ollama for local inference, local vector storage for RAG, and disabled-by-default background data updates.

Unlike the other requirements, privacy is a design constraint rather than a spectrum. Table 2.6 defines the verification checklist.

Table 2.6.: Verification checklist for R5: Privacy.

Privacy criterion	
☐	LLM inference runs locally via Ollama. No source code is sent to cloud APIs.
☐	RAG vector store and embeddings are stored locally. No code-derived data leaves the machine.
☐	Analysis prompts and results stay on localhost. No telemetry or external transmission.
☐	Optional vulnerability data refresh uses only public metadata (NVD, OWASP, CWE) and can be disabled for fully offline operation.
☐	No source code or code fragments are included in any outbound network request.

2.2. Related Work

This section reviews prior work on SAST, LLM-based security assistance, retrieval-augmented prompting, and privacy-preserving IDE integration, then positions the thesis approach against these paradigms.

2.2.1. Traditional Static Application Security Testing

Static Application Security Testing (SAST) tools remain the default workhorse for vulnerability detection in many development workflows. Rule-based analyzers and taint-analysis systems such as Semgrep and CodeQL statically inspect source code for predefined patterns and data-flow queries [18, 19]. Their main advantage is operational: results are deterministic, reproducible, and easy to integrate into CI/CD pipelines and compliance-driven environments.

From a research perspective, modern static analyzers build on foundational ideas such as abstract interpretation [20] and scalable bug-finding techniques [21]. In practice, large-scale deployments demonstrate that static analysis can find real defects in industrial codebases at massive scale [3]. Security-oriented static analysis has also been studied explicitly, including work that frames static analysis as an effective security engineering control when combined with secure development processes [4, 22].

A second advantage of SAST is *auditability*. Findings are tied to explicit rules or queries, which enables traceable reasoning, stable regression behavior, and predictable security gates. This is particularly relevant in regulated settings, where organizations

2. Analysis

need evidence of controls rather than only aggregate accuracy claims. Determinism also simplifies governance: when a rule is updated, teams can review the diff in findings directly instead of inferring changes from opaque model behavior.

These strengths come with well-known limitations. SAST tools can struggle with contextual reasoning and generalization, producing false positives when a risky-looking pattern is guarded elsewhere (e.g., validation in a different function, framework-enforced invariants) and false negatives when vulnerabilities depend on semantic relationships or framework-specific behavior. Language and framework abstractions further complicate exhaustive rule coverage and often push teams toward conservative, noisy rulesets.

There is also a structural precision/recall tension in static analysis deployments. Aggressive rule sets can improve vulnerable-case coverage, but often increase triage burden through noisy alerts; stricter rules reduce noise but risk under-detection. The trade-off is not only technical but organizational: teams with limited security-review capacity may prefer conservative rule sets, while high-assurance environments may tolerate larger alert queues to avoid misses.

Finally, many SAST tools provide limited remediation guidance, requiring developers to interpret findings and design fixes manually. Empirical studies show that warning overload and poor actionability reduce adoption and remediation rates [5, 6]. This gap between *detection* and *remediation support* is one reason why SAST outputs are often strongest as gatekeeping signals, but weaker as day-to-day developer coaching tools.

Table 2.7 evaluates the main SAST tools discussed above against the requirements defined in Section 2.1.

Table 2.7.: Evaluation of traditional SAST tools against requirements R1–R5.

Tool	R1	R2	R3	R4	R5	Key Gap
Semgrep [18]						No contextual reasoning or repair generation
CodeQL [19]						No repair suggestions; rigid query language
ESLint security plugins						Surface-level patterns only; no semantic analysis

All three tools achieve high consistency (R2) because their results are deterministic and reproducible. They also score high on usability (R4) due to fast execution and CI/CD integration, and high on privacy (R5) since they run fully locally. However, accuracy (R1) varies: CodeQL’s taint analysis provides better context than Semgrep’s

pattern matching, while ESLint security plugins cover only surface-level patterns. The main gap is repair quality (R3) — none of these tools suggest concrete fixes, leaving developers to interpret findings and write repairs manually.

These limitations motivate approaches that keep deterministic checks as guardrails while adding more flexible reasoning and repair generation closer to the developer workflow.

2.2.2. LLM-Based Vulnerability Detection and Repair

Recent advances in Large Language Models (LLMs) have renewed interest in using learned models for code understanding, vulnerability detection, and repair suggestions. Contemporary systems build on transformer architectures [23] and large-scale pretraining objectives in the style of BERT/T5 [24, 25]. Both general-purpose LLMs and code-specialized models can generate syntactically plausible code and perform transformations such as refactoring and patch drafting [7, 8]. For IDE-integrated security assistance, these capabilities are attractive because they enable explanations and candidate fixes at the point of code.

However, empirical evaluations show that model-generated code can be *functionally correct yet insecure*, and that probabilistic generation can introduce hallucinated assumptions or omit critical security checks [14, 13]. This matters disproportionately in security, where small omissions (e.g., missing validation, unsafe sinks) can create exploitable weaknesses.

Recent survey work shows both the breadth and the fragmentation of this area. Reviews by Zhang et al. and Gholami cover broad cybersecurity use cases, but both report recurring problems in reproducibility, evaluation quality, false positives, and operational trust [26, 27]. Focusing specifically on code security, Basic and Giarretta show that the literature spans not only detection and repair but also security risks introduced by LLM-generated code itself [28].

A key difference from classical static analysis is that LLM behavior is strongly prompt- and format-dependent. Small changes in instructions, output schema, or contextual examples can materially change both finding rates and false-positive behavior. As a result, measured model quality is not only a property of model weights, but also of engineering choices such as scope selection, schema strictness, retrieval context, and failure handling.

For IDE integration, this dependence has an additional implication: free-form natural language answers are often insufficient for automation. Practical tools typically require machine-checkable outputs (e.g., JSON findings with line spans) and conservative handling of malformed responses. In this thesis, structured-output robustness is therefore treated as a deployment gate, not merely a presentation detail.

2. Analysis

Before LLMs, machine-learning-based vulnerability detection explored learning semantic representations of code for classification. Early systems used deep learning over code fragments and patterns [29], while representation-learning work explored embeddings for code structure and semantics [30]. More recent approaches leveraged graph representations to capture richer program semantics [31, 32]. These methods improved over purely lexical baselines, but often required task-specific training data and did not naturally provide developer-facing explanations or repair suggestions.

Finally, security-related failure modes of LLMs extend beyond simple false positives/negatives. The broader LLM risk literature emphasizes both ethical/operational risks [33] and adversarial behaviors such as jailbreaks and prompt-based attacks [34]. For IDE-integrated security tools, these risks motivate strict output contracts, careful prompt design, and conservative integration of model suggestions into developer workflows.

Another practical challenge is *calibration*. Many LLM-based assistants can explain a vulnerability convincingly even when the underlying label is wrong or weakly grounded. Without explicit confidence controls and abstention behavior, this can increase developer over-trust. For security tooling, persuasive explanations are not sufficient; the system must also provide stable structure, traceable evidence, and clear boundaries between high-confidence findings and uncertain hypotheses.

Recent empirical studies sharpen these concerns. Li et al. show that context-poor evaluation can underestimate LLM capability and distort rationale assessment [35]. IRIS shows that LLMs are stronger when coupled with static analysis than when used alone [36]. CWEVAL shows that static-rule scoring can miss or misjudge functionally correct yet insecure outputs, motivating outcome-driven security oracles [37]. SecRepair similarly frames secure repair as a full-pipeline problem spanning data, model adaptation, and evaluation [38].

In response, contemporary approaches increasingly combine learned models with auxiliary signals such as retrieval, tools, or deterministic checks. Retrieval-augmented generation provides a mechanism to ground model reasoning in external security knowledge without retraining [39, 40]. Tool-oriented prompting (e.g., using dedicated retrieval or analysis tools) further motivates modular designs where different components provide complementary evidence [41]. In Code Guardian, these ideas are reflected in the optional RAG module and the strict JSON-output contract used for IDE diagnostics and evaluation.

Repair suggestion quality is also connected to the broader literature on automated program repair. Classic systems such as GenProg and subsequent patch-learning approaches show both the promise of automation and the difficulty of evaluating patch correctness beyond passing tests [42, 15]. For IDE-integrated security assistance, these findings motivate presenting repairs as *optional* quick fixes, requiring explicit developer review and preserving responsibility for functional correctness.

2. Analysis

Table 2.8 evaluates representative LLM-based approaches against the requirements defined in Section 2.1.

Table 2.8.: Evaluation of LLM-based vulnerability detection approaches against requirements R1–R5.

Approach	R1	R2	R3	R4	R5	Key Gap
GitHub Copilot [7]						Cloud-only; source code leaves the machine
IRIS [36]						Batch-only; no IDE integration or privacy
SecRepair [38]						No IDE workflow; cloud model assumed
CWEVAL [37]					—	Evaluation framework only; no repair or IDE

LLM-based approaches show stronger accuracy (R1) than SAST tools due to contextual reasoning, but consistency (R2) is weaker because of nondeterministic generation. GitHub Copilot offers the best usability (R4) through native IDE integration, but fails on privacy (R5) since it requires cloud inference. IRIS and SecRepair provide medium repair quality (R3) by combining static analysis with LLM-generated patches, but neither is integrated into IDE workflows (low R4). CWEVAL is an evaluation framework, not a repair tool, so R3 does not apply. The main gap across all approaches is the lack of local, privacy-preserving deployment.

2.2.3. Retrieval-Augmented Generation for Secure Coding

Retrieval-Augmented Generation (RAG) is a general paradigm for grounding language model outputs in external knowledge without retraining. In RAG, a retriever selects relevant documents for a given query and the generator conditions on those documents when producing an answer [39]. Retrieval can be implemented with lexical scoring functions such as BM25 [43] or with dense retrieval based on semantic embeddings. Dense retrieval approaches such as DPR and retrieval-augmented pretraining (REALM) motivate this design by showing that semantic retrieval can provide effective context for generation [40, 44]. Survey work further highlights RAG as a practical way to reduce hallucination and improve factuality [45, 13].

In secure coding settings, the retrieved context typically corresponds to vulnerability taxonomies (e.g., CWE), secure coding guidelines (e.g., OWASP cheat sheets), and historical vulnerability examples. This fits vulnerability detection and repair well

2. Analysis

because many issues are best explained by mapping code patterns to known weakness classes and established mitigations [2, 1]. By grounding the model in such sources, a system can improve detection consistency (R2) and explanation quality (R4), while reducing unsupported guesses.

Recent secure-code generation research suggests that *what* is retrieved matters as much as retrieval itself. RESCUE argues that raw security documents are often too noisy; it instead distills vulnerability-fix data into hierarchical guidance and concise code examples, then retrieves across security facets such as API pattern, vulnerability cause, and code structure [46]. The same lesson is relevant for vulnerability detection: curated security-specific retrieval units are likely to be more useful than generic prose.

Nevertheless, RAG introduces its own failure modes. If retrieval returns weakly related snippets, the generator can become distracted and produce confident but misaligned explanations. If retrieval returns overly generic security text, outputs may drift toward checklist-style warnings that increase false positives. In security tooling, retrieval quality is therefore as important as model quality: grounding helps only when retrieved context is both relevant and specific to the analyzed code pattern.

Implementation-wise, RAG depends on embedding models and efficient nearest-neighbor search. Sentence-level embedding methods such as Sentence-BERT are commonly used to map text into vector space [47]. Approximate nearest-neighbor indices such as HNSW provide high recall with practical latency [48], and libraries such as FAISS popularized large-scale similarity search in practice [49]. In Code Guardian, these ideas are realized through local embeddings and a persistent HNSW vector store to keep retrieval on-device and compatible with IDE latency constraints.

From an engineering perspective, RAG design involves a three-way trade-off among latency, grounding depth, and noise:

- increasing top- k retrieved chunks can improve recall of relevant guidance but expands prompt size and latency;
- larger chunks preserve context but may dilute precision in similarity search;
- aggressive knowledge refresh increases topicality but can introduce quality variance and reduce reproducibility if source curation is weak.

These trade-offs motivate model-specific calibration rather than one global RAG configuration. The same retrieval setup can improve one model while degrading another, depending on how each model integrates external context under strict output constraints. This observation is central to the ablation design in this thesis.

Table 2.9 evaluates representative RAG-based approaches against the requirements defined in Section 2.1.

Table 2.9.: Evaluation of RAG-based secure coding approaches against requirements R1–R5.

Approach	R1	R2	R3	R4	R5	Key Gap
Generic RAG [39]					—	No IDE integration; privacy depends on deployment
RESCUE [46]					—	No IDE or usability focus; cloud evaluation only

RAG improves accuracy (R1) by grounding LLM reasoning in security knowledge, though the degree of improvement is model-dependent. RESCUE achieves higher repair quality (R3) than generic RAG because it retrieves structured vulnerability-fix pairs rather than raw security text. Consistency (R2) benefits from retrieval grounding but remains medium since the underlying LLM is still nondeterministic. Neither approach reports usability metrics for IDE integration (R4), and privacy (R5) depends on deployment — generic RAG can run locally but is often evaluated with cloud models. The key insight is that RAG is a useful augmentation, not a standalone solution.

2.2.4. Privacy-Preserving and IDE-Integrated Approaches

Privacy concerns pose a significant barrier to adopting LLM-based security tools in real-world settings. Cloud-hosted assistants require transmitting proprietary source code to external servers, which is unacceptable in many regulated or security-sensitive environments [10]. Beyond policy constraints, research has demonstrated concrete privacy risks in large language models, including memorization and extraction of training data [9] and inference attacks that raise questions about model confidentiality and data exposure [50]. For security tooling, these risks motivate designs that keep source code and analysis artifacts local by default.

The IDE context introduces additional security considerations. Because the assistant processes attacker-controlled inputs (e.g., comments, strings, dependency code), prompt-injection and tool-manipulation attacks become relevant; OWASP explicitly catalogs such risks for LLM applications [12]. These concerns motivate designs that treat code as data, constrain outputs to machine-checkable schemas, and isolate retrieval sources to curated security knowledge.

IDE-integrated security tools can improve developer engagement and remediation rates by providing feedback during active development rather than post hoc analysis [5, 6]. However, many IDE assistants prioritize productivity features such as code completion and refactoring, with limited focus on security guarantees, reproducibility,

2. Analysis

or privacy boundaries. In practice, local deployment and deterministic interaction contracts become critical: users must understand what data is processed, where it is processed, and how results may vary across models and runs.

Local deployment, however, is not a complete security solution. Moving inference on-device shifts the trust boundary from cloud providers to local runtime integrity, workstation hardening, and extension-level data handling. Attackers who gain local access can still exfiltrate prompts, caches, or generated repairs unless operational controls (OS hardening, access control, and secure storage defaults) are in place.

A second practical boundary concerns *mixed connectivity*. Many systems operate with local inference but optional online knowledge refresh. This hybrid pattern can preserve source-code privacy while still introducing supply-chain and provenance risks if external knowledge sources are not validated. Privacy-preserving design should therefore separate code-bearing data paths from public-metadata update paths and make this separation visible to users.

Table 2.10 evaluates representative privacy-aware and IDE-integrated approaches against the requirements defined in Section 2.1.

Table 2.10.: Evaluation of privacy-preserving and IDE-integrated approaches against requirements R1–R5.

Approach	R1	R2	R3	R4	R5	Key Gap
GitHub Copilot [7]						No privacy; all code sent to cloud
Snyk Code (local) [?]]						No repair suggestions; partial cloud dependency
Semgrep (local) [18]						No contextual reasoning or repair generation

GitHub Copilot offers the best IDE experience (R4) and medium repair quality (R3), but fails on privacy (R5) because all inference happens in the cloud. Snyk Code’s local mode provides high consistency (R2) and good usability (R4), but its proprietary engine limits transparency and it offers no repair suggestions (low R3). Its privacy score is medium-high because some features still require cloud connectivity. Semgrep runs fully locally (high R5) with deterministic results (high R2), but lacks contextual reasoning and repair generation. The gap across all approaches is the combination of local privacy, contextual LLM reasoning, and repair suggestions in a single IDE-integrated tool.

2.2.5. Positioning of This Thesis

In contrast to much prior work, this thesis focuses on privacy-preserving vulnerability detection and repair using locally deployed LLMs integrated into Visual Studio Code. The proposed system combines retrieval-augmented reasoning with IDE-level context extraction to support both real-time and on-demand security analysis for JavaScript and TypeScript codebases. Unlike fine-tuned or cloud-dependent approaches, the system runs on-device by default and can operate fully offline when knowledge refresh is disabled. It also decouples security knowledge from model parameters and is evaluated through documented local ablation experiments (LLM-only vs. LLM+RAG) with quantitative metrics.

Comparison with Cloud-Based Code Assistants. GitHub Copilot and Amazon CodeWhisperer represent the dominant paradigm for AI-assisted coding, including security-related suggestions [7?]. These systems leverage large-scale cloud infrastructure to provide real-time code completions and, increasingly, vulnerability scanning capabilities. GitHub Advanced Security integrates Copilot-powered code scanning with secret detection and dependency review [?]. However, both approaches require transmitting source code to external servers for inference, which is unacceptable in regulated industries, government projects, or environments with strict intellectual property controls. Code Guardian addresses this limitation by keeping all inference local, enabling security assistance for codebases that cannot leave organizational boundaries.

Comparison with Hybrid SAST Tools. Modern SAST vendors increasingly offer hybrid modes. Snyk Code provides both cloud-based and local analysis options, with the local mode running a proprietary semantic analysis engine on-device [?]. Semgrep supports fully local rule-based scanning with optional cloud features for team management and custom rule sharing [18]. Unlike these tools, Code Guardian combines local LLM inference with retrieval-augmented prompting, enabling contextual reasoning and natural-language explanations that pure rule-based systems cannot provide. The trade-off is that LLM-based analysis introduces probabilistic behavior and higher latency compared to deterministic SAST, which motivates the hybrid deployment strategies discussed in Chapter 5.

Comparison with Research Prototypes. Academic systems such as IRIS [36] and SecRepair [38] demonstrate strong results by combining static analysis with LLM reasoning. However, these systems are typically evaluated in batch mode on benchmark datasets rather than integrated into interactive IDE workflows. They also often assume access to cloud-hosted models or require significant computational resources. Code Guardian prioritizes deployability on developer workstations with

2. Analysis

consumer-grade hardware, treating IDE integration latency and structured-output robustness as first-class requirements rather than secondary implementation details.

Differentiation Summary. Table 2.11 summarizes the key differences between Code Guardian and representative alternatives across the dimensions most relevant to this thesis.

Table 2.11.: Positioning comparison with representative security assistance approaches.

Approach	Privacy	Contextual reasoning	Repair sug- gestions	IDE integra- tion	Output con- tract
GitHub Copilot	Cloud-dependent	Strong (LLM)	Yes	Native	Unstructured
Snyk Code (local)	Local analysis	Rule-based	Limited	Plugin	Structured
Semgrep	Fully local	Rule-based	No	CLI/Plugin	Structured
IRIS / SecRepair	Varies	Strong (hybrid)	Yes	Batch-only	Research-grade
Code Guardian	Fully local	LLM + RAG	Yes	Native VS Code	Strict JSON

Table 2.12 provides a unified requirement-level comparison across all reviewed approaches using the evaluation scales from Section 2.1. This table consolidates the per-subsection evaluations into a single view, making it possible to identify the specific requirement gaps that Code Guardian addresses.

Table 2.12.: Unified requirement comparison of all reviewed approaches and Code Guardian.

Approach	R1	R2	R3	R4	R5
Semgrep					
CodeQL					
GitHub Copilot					
IRIS					
SecRepair					
RESCUE					—
Snyk Code					
Code Guardian					

The table shows that no existing approach satisfies all five requirements at medium or above. SAST tools excel at R2, R4, and R5 but lack contextual reasoning (R1) and repair generation (R3). LLM-based research prototypes offer strong R1 and R3 but fail on R4 (no IDE integration) and R5 (cloud dependency). Cloud assistants like Copilot trade privacy for usability. Code Guardian is the only approach that achieves at least medium on all five requirements simultaneously, addressing the combined gap that motivates this thesis.

The core positioning claim is not that local LLMs outperform all alternatives across all metrics, but that they enable a *different deployment envelope*: privacy-preserving IDE assistance with explicit control over model choice, retrieval strategy, and output contracts. This makes the approach operationally distinct from cloud assistants and complements pure SAST pipelines rather than replacing them.

Concrete Contributions Relative to Prior Work. Relative to existing work, this thesis contributes in five concrete ways.

1. **Structured-output robustness as a first-class requirement.** Many LLM-based security tools produce free-form natural language that requires manual interpretation. Code Guardian enforces a strict JSON contract with format-level enforcement and defensive parsing to enable automated diagnostics and quick-fix generation.

2. **Multi-pass consensus filtering for false-positive reduction.** The system runs analysis twice with different seeds and keeps only findings confirmed by both passes. This directly reduces false positives because inconsistent findings are filtered out, while true positives reproduce reliably. No reviewed approach applies this technique to LLM-based vulnerability detection.
3. **Deterministic inference controls.** The system sets temperature to zero with a fixed seed to maximize reproducibility across runs. Combined with JSON-mode decoding, this addresses a key weakness of LLM-based tools: inconsistent output across identical inputs.
4. **RAG as a model-dependent intervention.** The evaluation explicitly measures which models benefit from retrieval augmentation and which degrade, rather than assuming universal improvement. This provides actionable guidance for configuration selection.
5. **Unified local evaluation harness.** The system is compared against established SAST tools (Semgrep, CodeQL, ESLint) under the same local evaluation harness and dataset, enabling direct interpretation of quality-versus-noise trade-offs in a shared setup.

By addressing accuracy, consistency, repair quality, usability, and privacy within a unified framework, this work bridges the gap between academic advances in LLM-based security analysis and the practical requirements of real-world development workflows.

2.2.6. Critical Comparison Across Paradigms

The reviewed literature indicates that security assistants should be compared as *design paradigms*, not as isolated tools. Table 2.13 summarizes the trade-offs that motivate this thesis architecture.

2. Analysis

Table 2.13.: Critical comparison of major security-assistance paradigms.

Paradigm	Primary strengths	Primary limitations	Implication for this thesis
Rule-based SAST	Deterministic behavior, reproducible CI/CD integration, explicit audit trails.	Limited semantic adaptation and weak repair guidance.	Keep deterministic baseline signals and add contextual reasoning.
LLM-only IDE assistant	Flexible reasoning, explanation, and repair suggestion in context.	Probabilistic behavior, hallucination risk, and variable false-positive control.	Enforce strict output contracts and developer-controlled patching.
RAG-assisted LLM	Better grounding and explanation alignment in favorable setups.	Model-dependent gains and additional latency/-complexity.	Treat RAG as tunable per model/workflow, not universal.
Local-first deployment	Strong source-code privacy boundary and reproducibility.	Dependence on local hardware/host security assumptions.	Prioritize no-exfiltration while documenting residual risks.

Two conclusions follow. First, no single paradigm satisfies R1–R5 alone; practical systems combine complementary mechanisms. Second, evaluation must jointly report quality and operational metrics because deployment value depends on recall, false-positive control, repair quality, latency, and privacy guarantees together.

2.3. Summary

This chapter established the requirements baseline for the thesis and compared the main design paradigms available in the literature. The analysis shows that a practical solution must combine deterministic baselines, contextual reasoning, explicit privacy boundaries, and workflow-aware responsiveness. These conclusions motivate the concept presented in the next chapter.

3. Concept

This chapter translates the five requirements from Chapter 2 into the conceptual design of Code Guardian — a privacy-preserving, IDE-integrated system for detecting and repairing security vulnerabilities in JavaScript and TypeScript code using locally deployed LLMs with optional retrieval augmentation.

The design is motivated by three shortcomings in existing work: traditional SAST tools rely on predefined rules and struggle with contextual reasoning [4, 3]; cloud-based LLM assistants require transmitting source code to external servers [10, 51]; and current approaches provide limited repair guidance [5, 6]. Code Guardian addresses these limitations through a modular architecture that directly supports the five requirements: multi-pass consensus filtering for detection accuracy (R1), JSON-mode decoding and deterministic inference for consistency (R2), a two-stage detection-then-repair pipeline for actionable remediation (R3), debouncing and caching for IDE-responsive latency (R4), and local-only inference to preserve source-code privacy (R5).

Section 3.1 derives the architectural decisions from the analysis results. Section 3.2 describes the core components. Section 3.3 presents the detection workflows. Section 3.4 provides C4 architecture views and sequence diagrams. Implementation and evaluation are deferred to Chapters 4 and 5.

3.1. Concept Derivation from Analysis Results

The conceptual design of the proposed vulnerability detection system is derived systematically from the five requirements identified in Section 2.1 and the gaps observed in existing approaches reviewed in Section 2.2. This section traces how each architectural decision directly addresses specific limitations in current vulnerability detection systems, establishing the rationale for a local, RAG-augmented, IDE-integrated design.

3.1.1. From Cloud-Based to Privacy-Preserving Local Deployment

The analysis in Section 2.2 revealed that most LLM-based vulnerability detection systems rely on cloud-hosted models or external APIs, requiring transmission of source code to remote servers. Regulatory frameworks such as GDPR impose strict constraints on transmitting sensitive data to external services, and the OWASP LLM Top 10 explicitly lists sensitive information disclosure as a key risk for LLM-based applications [10, 12]. Research has further shown that code generation models can leak sensitive information from their training data, including proprietary code submitted through cloud-based coding assistants [9, 51]. Even without direct leakage, membership inference attacks can reveal whether specific code fragments were included in a model’s training set [50].

These observations directly motivate the decision to deploy all components locally within the developer’s environment. By running LLMs, retrieval systems, and analysis components entirely on local hardware, the design aims to keep source code, intermediate representations, and analysis results on-device and to avoid external inference services. This local-first architecture supports R5 (privacy) and enables offline operation when knowledge refresh is disabled, making the system suitable for security-sensitive development environments.

In the Code Guardian prototype, the privacy boundary is defined around *source code*: analyzed code and IDE-derived context are sent only to a local LLM backend. The system may optionally refresh its *security knowledge base* from public vulnerability metadata sources (e.g., CVE/NVD descriptions and references). Such refresh operations do not transmit user source code and can be disabled to operate fully offline with cached or baseline knowledge.

The trade-off for local deployment is increased latency compared to cloud-based systems with dedicated accelerators. However, by carefully optimizing model selection, retrieval strategies, and context management, the system can achieve acceptable response times on standard developer hardware, addressing R4 (usability).

3.1.2. Retrieval-Augmented Generation for Security Knowledge Grounding

Requirements R1 and R2 demand accurate and consistent vulnerability detection across repeated analyses. Pure generative LLM approaches can suffer from hallucination and inconsistent classifications [13]. R1 also requires context-aware reasoning that goes beyond surface-level pattern matching to understand data flow, control flow, and API semantics.

3. Concept

The system addresses these requirements through Retrieval-Augmented Generation (RAG), which grounds vulnerability detection in a locally maintained knowledge base of security information [39, 40]. This knowledge base includes:

- **CWE-aligned vulnerability patterns** and mitigation summaries [2]
- **OWASP guidance** for recurring web vulnerability categories [1, 52]
- **CVE/NVD summaries** (public descriptions and references) to keep the knowledge base current [53, 54]
- **Curated JavaScript/TypeScript security notes** (e.g., prototype pollution, unsafe deserialization patterns)

During analysis, relevant security knowledge is retrieved based on semantic similarity to the code under examination and provided as context to the LLM. This design reduces reliance on purely generative reasoning and enables the knowledge base to evolve independently of model parameters. By decoupling security knowledge from the model, the system supports continuous updates to vulnerability information without requiring model retraining.

However, retrieval augmentation is not universally beneficial. As demonstrated in the evaluation (Chapter 5), RAG improves detection quality for some model families but degrades it for others. This model-dependent effect means RAG should be treated as a configurable option rather than a default assumption.

3.1.3. IDE Integration for In-Context Security Assistance

Traditional SAST tools operate as separate processes in CI/CD pipelines, providing feedback only after code is committed. This delayed feedback loop increases cognitive load and reduces remediation rates, as developers must context-switch between writing code and reviewing security findings [5, 6].

The proposed system integrates directly into Visual Studio Code as an extension, providing security analysis during active development. This design supports two interaction modes:

- **Inline detection:** Debounced vulnerability annotations triggered by document changes, providing IDE-native feedback during active development
- **On-demand analysis:** Explicit analysis commands for comprehensive security audits of selected code regions or entire files

IDE integration directly addresses R4 (usability) by enabling rich, interactive presentation of vulnerability findings, explanations, and repair suggestions within the developer’s familiar environment. It also supports R3 (repair quality) by allowing developers to preview, review, and apply suggested fixes with minimal friction.

3.1.4. Modular Component Architecture

The analysis in Section 2.2 showed that monolithic vulnerability detection systems lack transparency: when detection fails or produces unexpected results, it is difficult to diagnose whether the error originated in context extraction, retrieval, reasoning, or explanation generation.

The proposed system decomposes vulnerability detection into four core components, each with clearly defined responsibilities:

1. **Context Extraction:** Determines analysis scope (enclosing function, selection, file) and produces snippet text plus localization metadata
2. **Knowledge Retrieval:** Queries the local security knowledge base to retrieve relevant vulnerability information
3. **Vulnerability Detection:** Analyzes code context using the LLM, grounded in retrieved security knowledge
4. **Repair Generation:** Produces developer-controlled repair suggestions as optional patches (quick fixes)

This modular design enables component-level analysis, isolated improvement, and transparent error diagnosis. Each component can be tested, optimized, and replaced independently without destabilizing the overall architecture. Intermediate outputs (extracted context, retrieved knowledge, detection reasoning) remain visible for debugging and validation, supporting transparency and reproducibility.

3.1.5. Context-Aware Analysis with Code Understanding

Requirement R1 demands that the system correctly identify vulnerabilities based on semantic and structural context rather than syntactic patterns alone. Surface-level pattern matching produces false positives when secure code resembles vulnerable patterns, and false negatives when vulnerabilities depend on data flow or control flow relationships.

The system addresses this requirement primarily through *scope selection* and *prompt grounding*. In the current prototype, context extraction focuses on reliably selecting a coherent code region (e.g., the enclosing function) and mapping findings back to the editor. The LLM is then expected to reason about data flow and control flow *within the provided scope*, optionally grounded by retrieved security guidance.

Conceptually, context-aware reasoning benefits from multiple kinds of context, including:

- **Syntactic context:** function boundaries and code structure derived from AST parsing (used for scoping)

3. Concept

- **Local semantic context:** identifiers, API calls, and validation logic present in the analyzed region
- **Inferred data/control flow:** reasoning about sources, sinks, and guards within the snippet

This approach supports R1 for many localized vulnerability patterns, but it has inherent limitations when required evidence lies outside the analyzed scope (e.g., validation in a different file). Chapter 6 outlines extensions such as explicit taint-style traces and cross-file context extraction to address these cases.

3.1.6. Explainability Through Structured Reasoning

Transparent explanations that link detected vulnerabilities to concrete code regions and recognized security principles are essential for usability (R4) and repair quality (R3). Opaque "black box" detection undermines developer trust and makes it difficult to validate findings or apply fixes correctly [5, 6].

The system enforces explainability through structured reporting and IDE-native presentation. In the diagnostics pipeline, vulnerability reports include:

- **Code localization:** Specific lines or code fragments contributing to the vulnerability
- **Issue explanation:** A short message describing why the code is risky and what pattern is being flagged
- **Optional repair:** A suggested patch presented as a quick fix (developer-controlled)

In interactive webview modes, the system can provide longer, Markdown-formatted explanations, and (when enabled) can incorporate retrieved security knowledge from sources such as CWE and OWASP into the prompt to ground the reasoning [2, 52]. The evaluation harness used in Chapter 5 additionally requests explicit **type** and **severity** fields to support quantitative scoring.

By grounding explanations in retrieved security knowledge (when used) and enforcing structured output formats where needed, the system produces actionable vulnerability reports that are auditable at the IDE level. This design improves developer trust by making detection outputs inspectable and reviewable, supporting both usability (R4) and repair quality (R3).

3.1.7. Deterministic Inference and Consensus-Based Detection

A key challenge for LLM-based detection is output instability: the same code analyzed twice may yield different findings, undermining developer trust and complicating evaluation (R2). Two complementary mechanisms address this.

Deterministic inference controls. The system configures the LLM backend with temperature 0 and a fixed random seed to maximize reproducibility across runs. Additionally, JSON-mode decoding (`format: 'json'`) is enabled at the Ollama API level, which constrains the model's token sampling to produce only valid JSON tokens. This eliminates parse failures caused by malformed output and ensures that every response is machine-readable without post-hoc repair.

Multi-pass consensus filtering. To reduce false positives (R1), the system runs the detection prompt twice with different random seeds (e.g., seed 42 and seed 137). Findings are compared across passes using vulnerability-type and line-range matching. Only issues confirmed by both passes are retained in the final result. This consensus mechanism filters out spurious detections that arise from sampling variance, improving precision without sacrificing recall on consistently detected vulnerabilities. If consensus filtering removes all findings, the system falls back to the first pass result to avoid silent suppression.

3.1.8. Two-Stage Detection and Repair Pipeline

Existing approaches typically generate detection findings and repair suggestions in a single prompt, which forces the model to reason about both tasks simultaneously. This coupling can degrade both detection precision and repair quality, because the model may anchor on generating a plausible fix rather than accurately classifying the vulnerability.

Code Guardian separates these concerns into two stages:

1. **Stage 1 — Detection:** The detection prompt asks the model to identify vulnerabilities and return structured findings (type, severity, location, description). No repair is requested at this stage.
2. **Stage 2 — Repair:** For each confirmed vulnerability from Stage 1 (after consensus filtering), a dedicated repair prompt provides the vulnerability type, affected lines, and full code context. The model generates a targeted fix for the specific issue.

This separation allows each prompt to focus on a single task, improving both detection accuracy (R1) and repair quality (R3). The two-stage design also enables parallel repair generation when multiple vulnerabilities are detected, since each repair prompt is independent.

Each architectural decision in this section is tied to the requirements established in Chapter 2. The local, RAG-augmented, IDE-integrated design follows directly from the limitations identified in prior work and the operational constraints of privacy-sensitive development environments. The next section details the core components that realize this design, including their inputs, outputs, and processing logic.

3.2. System Components

The Code Guardian prototype is organized into a small set of components that map directly to the requirements from Chapter 2. The system is implemented as a VS Code extension that (i) extracts an appropriate analysis scope from the editor, (ii) invokes a local LLM for vulnerability analysis, (iii) optionally augments prompts with locally retrieved security knowledge (RAG), and (iv) renders findings and fix suggestions using IDE-native UI elements.

3.2.1. Context Extraction Component

The context extraction component determines *which* code is analyzed for a given interaction mode and provides the analyzer with enough metadata to localize findings in the editor.

Scopes. Code Guardian supports multiple scopes with different latency and completeness characteristics:

- **Function scope (real-time)** extracts the innermost function-like block at the cursor position (function declarations, arrow functions, methods, constructors). This is the default for continuous feedback because it keeps the prompt small.
- **File scope (on-demand)** analyzes the full current document and returns a comprehensive set of findings as diagnostics.
- **Selection scope (interactive)** sends a selected region (or current line) into a webview-based analysis view that supports follow-up questions.
- **Workspace scope (batch)** scans all JS/TS files and aggregates results into a security dashboard.

AST-based function extraction. Function scope extraction is implemented by parsing the current document with the TypeScript compiler API and selecting the smallest enclosing `FunctionLikeDeclaration` around the cursor. The extractor returns both the extracted snippet and its start-line offset (0-based) in the original document. This offset enables precise mapping of model-reported line numbers back to VS Code diagnostic ranges.

Design rationale. Function scoping is a practical mechanism to keep latency predictable for real-time use (R4), while the line-offset mapping improves usability by ensuring that findings point to the correct source locations. Deeper program context (imports, cross-file flows) is not extracted explicitly in the current prototype and is treated as a key future-work direction.

3.2.2. Knowledge Retrieval Component (RAG)

The retrieval component implements Retrieval-Augmented Generation (RAG) to ground LLM reasoning in local security knowledge [39, 40]. In Code Guardian, retrieval is implemented by a dedicated **RAGManager** that maintains a local knowledge base and a persistent vector index.

Knowledge base contents. The knowledge base is populated from a mix of curated and fetched *public* security metadata:

- **CWE-aligned entries** describing common weakness patterns and mitigations [2].
- **OWASP Top 10 guidance** for recurring web vulnerability categories [1].
- **CVE/NVD summaries** retrieved from the NVD API (descriptions and references) [54, 53].
- **JavaScript ecosystem advisories** for dependency and platform-specific risks.

Knowledge artifacts are cached on disk and reused across sessions. When network access is unavailable, the system falls back to a small baseline knowledge bundle so retrieval remains functional (with reduced coverage).

Indexing and retrieval. Knowledge entries are chunked using a recursive splitter (chunk size 1000, overlap 200) and embedded locally through an Ollama-served embedding model. Embeddings are stored in a local HNSW vector index [48] via LangChain tooling [55]. At query time, the top- k most similar chunks are retrieved (runtime default $k = 3$; the evaluation harness in Chapter 5 uses $k = 5$) and injected into the prompt as an explicit “relevant security knowledge” section.

Privacy boundary. Retrieval and embeddings are executed locally. Optional knowledge refresh operations fetch only public vulnerability metadata; user source code is not transmitted to those endpoints (R5).

3.2.3. Vulnerability Detection Component

The vulnerability detection component performs local LLM inference and returns findings in a format suitable for IDE integration.

Structured-output analyzer for diagnostics. For inline diagnostics, Code Guardian enforces structured output through two mechanisms: a strict JSON-only output contract in the system prompt, and JSON-mode decoding at the Ollama API level (`format: 'json'`), which constrains token sampling to valid JSON. Deterministic inference (temperature 0, fixed seed) further improves consistency (R2). Each finding includes:

- **message:** a short description of the issue.
- **startLine/endLine:** 1-based line indices relative to the analyzed snippet.

Repair suggestions are not requested during detection. They are generated separately in Stage 2 of the two-stage pipeline (Section 3.2.4) and attached to confirmed findings before rendering. If no issues are found, the model must return an empty array `[]`. The analyzer performs defensive parsing by stripping code fences and extracting the first JSON array substring if the model emits additional text.

Multi-pass consensus filtering. To reduce false positives (R1), the detection component runs the analysis prompt twice with different random seeds. Only findings confirmed by both passes—matched by vulnerability type and line range—are retained. This filters out spurious detections caused by sampling variance without requiring additional model capacity.

Failure handling and caching. Transient inference failures (timeouts, local server warm-up) are handled through retry with exponential backoff. To reduce repeated inference on unchanged code, results are cached with an LRU-style strategy keyed by a hash of (code, model, prompt mode), improving responsiveness during iterative edits.

Interactive analysis mode. In addition to the structured diagnostics flow, Code Guardian includes a webview-based analysis view for selected code. This mode uses Markdown-formatted responses and supports follow-up Q&A. When enabled, the RAG manager can be used to enrich the system prompt and user prompt for this interactive workflow.

3.2.4. Repair Generation Component

Repair generation follows a **two-stage pipeline** that separates detection from repair. In Stage 1, the detection prompt identifies vulnerabilities without requesting fixes. In Stage 2, each confirmed vulnerability (after consensus filtering) is sent to a dedicated repair prompt that receives the vulnerability type, affected lines, and full code

3. Concept

context. This separation allows the repair model to focus on generating a targeted fix for a specific, validated issue rather than simultaneously reasoning about detection and remediation.

The repair prompt returns a JSON object containing the suggested fix code. The IDE integration layer exposes each repair as a quick fix action. Applying a fix is always user-initiated and can be reverted via the editor undo stack (R3). The system does not automatically validate functional correctness of repairs; this is treated as a future improvement area.

3.2.5. IDE Integration Layer

The IDE integration layer connects the analysis pipeline to VS Code’s APIs [56] and determines when analyses run and how results are presented.

Triggers. The extension supports both automatic and manual triggers:

- **Debounced real-time analysis** on document changes (default debounce: 800 ms) for JavaScript/TypeScript documents.
- **Manual file analysis** via a command palette command, with a size guard to avoid excessively large prompts.
- **Interactive selection analysis** and **contextual Q&A** views implemented as WebViews.
- **Workspace scanning** that aggregates results into a dashboard view.

Presentation. Findings are rendered as VS Code diagnostics (squiggles, Problems panel, hover tooltips). Suggested repairs are attached to diagnostics and exposed as a quick fix action.

3.2.6. Supporting Subsystems

Several additional subsystems are important for usability and reproducibility:

- **Model management** queries available local Ollama models, filters for suitable code models, and supports runtime model switching.
- **Analysis cache** is a bounded LRU cache (100 entries, 30-minute TTL) with user-visible hit/miss statistics.
- **Workspace dashboard** computes a coarse security score using issue density and keyword-based severity heuristics and visualizes the distribution across files.

- **Vulnerability data manager** caches public metadata (e.g., OWASP/CWE/CVE-derived entries) with a time-based expiry to support offline reuse after updates.

This section outlined the core components of Code Guardian. The next section explains how these components are orchestrated into workflows for real-time feedback, on-demand inspection, and batch scans.

3.3. Detection Workflows

While the component architecture defines what responsibilities exist in the system, IDE usability depends on how these components are orchestrated in concrete workflows. Code Guardian supports several workflows that trade off latency, context breadth, and output structure. This section describes the main workflows implemented in the prototype and relates them to requirements R1–R5.

3.3.1. Real-Time Function-Level Diagnostics

The real-time workflow provides continuous feedback while a developer edits JavaScript/TypeScript code. The key goal is to deliver timely, IDE-native warnings without disrupting flow (R4).

Trigger. The extension listens to document change events for JavaScript and TypeScript documents. Analysis is *debounced* (default: 800 ms) so the model is not invoked on every keystroke.

Scope and guards. When the debounce fires, Code Guardian extracts the innermost enclosing function at the cursor position and analyzes only that snippet. A size guard skips unusually large functions (default: 2000 characters) to bound worst-case latency and avoid overloading the local model.

Structured output for diagnostics. The analyzer is prompted to return a strict JSON array of issues. Each issue contains a message and a line range. Repair suggestions are generated separately in Stage 2 and attached before rendering. The diagnostic adapter maps the snippet-relative, 1-based line numbers to VS Code ranges using the extractor’s line offset and clamps ranges to valid document bounds. This enables stable rendering in the Problems panel, editor squiggles, and hover tooltips.

Trade-offs. Function-level analysis improves responsiveness, but it can miss vulnerabilities whose evidence lies outside the current scope (e.g., validation in a different module). This limitation motivates the on-demand and workspace workflows and is addressed further in the future-work chapter.

3.3.2. On-Demand File Diagnostics

The file workflow provides broader coverage when a developer explicitly requests a deeper scan.

Trigger. A command palette action runs analysis over the full active document.

Scope guard. To avoid generating excessively large prompts, the prototype skips very large files (default: 20,000 characters) and warns the user instead.

Output. Findings are surfaced through the same structured diagnostics pipeline as the real-time workflow. This keeps the UI consistent, and it ensures that file scans can be reviewed in the Problems panel and navigated using standard IDE tooling.

3.3.3. Interactive Analysis and Follow-up Q&A

Some security questions require richer explanations than a single diagnostic message. Code Guardian therefore includes webview-based interaction modes.

Selection analysis view. The user can analyze a selected region (or the current line) and inspect the response in a dedicated analysis panel. This mode supports follow-up questions and streams Markdown-formatted responses for readability. Because it is user-initiated and not continuously triggered, it can tolerate higher latency than real-time diagnostics.

Contextual Q&A view. The contextual Q&A panel allows the user to select files and folders as context and ask security questions about that subset of the workspace. The extension reads the selected files locally and sends their contents only to the local LLM backend, preserving the no-exfiltration objective for source code.

RAG usage. When enabled, the RAG manager can enrich prompts for these interactive workflows by injecting retrieved security knowledge snippets (CWE/OWASP/CVE-derived guidance). Retrieval and embeddings are performed locally; optional knowledge refresh operations fetch only public metadata.

3.3.4. Workspace Scan and Security Dashboard

The workspace workflow supports periodic audits and prioritization across a repository.

File discovery and bounds. The scanner enumerates `*.js`, `*.jsx`, `*.ts`, and `*.tsx` files in the workspace while excluding dependency folders such as `node_modules`. Very large files are skipped (default: >500 KB) to keep runtime bounded.

Aggregation and scoring. Scan results are aggregated into a dashboard WebView. In addition to listing per-file issues, the dashboard computes a coarse security score

3. Concept

based on issue density (issues per KLOC) and a keyword-based severity heuristic. This score is intended for prioritization rather than as a formal risk metric.

Developer interaction. The dashboard supports opening affected files directly from the report and rerunning scans. This workflow complements real-time diagnostics by helping developers understand which parts of the codebase concentrate the most findings.

3.3.5. Repair Suggestion Workflow

After consensus-filtered detection (Stage 1), the system generates targeted repairs for each confirmed vulnerability using a dedicated repair prompt (Stage 2). Repairs are exposed as VS Code quick fixes. Applying repairs is always user-initiated and integrates with the undo stack. This preserves developer control (R3) and reduces the risk of unintended behavioral changes from automatically applied patches.

3.3.6. Workflow Selection in Practice

In typical use, workflows complement each other:

1. Developers receive continuous feedback via debounced, function-level diagnostics while writing code.
2. Before committing or during review, developers run file scans for broader coverage.
3. For ambiguous findings or design-level questions, developers use the interactive analysis and contextual Q&A views to obtain richer explanations and mitigation guidance.
4. Periodically, workspace scans provide an aggregate view that supports prioritization and remediation planning.

Together, these workflows operationalize the core idea of Code Guardian: privacy-preserving local analysis combined with IDE-native feedback and developer-controlled remediation.

3.4. System Architecture and Design

This section presents the high-level architecture of Code Guardian using C4-style views (context, containers, and process). The goal is to make the privacy boundary and the main data flows explicit: code stays on-device, local inference is performed

through a localhost LLM backend, and retrieval (when enabled) is backed by a local knowledge base and vector index.

3.4.1. Context View: System Boundary and External Interactions

Figure 3.1 positions Code Guardian within its operational environment. The primary actor is the developer working inside Visual Studio Code. The system executes in the VS Code extension host and communicates with a local LLM backend (Ollama) running on the same machine [11].

Local assets. The core local assets are:

- **Workspace source code** (JavaScript/TypeScript) being edited.
- **Local LLM runtime** (Ollama) used for analysis and (optionally) embeddings.
- **Local security knowledge base** and **vector index** used for retrieval when RAG is enabled [55, 48].
- **Local caches** for analysis results and vulnerability metadata.

Optional external data. Code Guardian may optionally fetch *public vulnerability metadata* to refresh the knowledge base (e.g., CVE records from the NVD API). This traffic does not include user source code and can be disabled by running in offline mode. After a refresh, cached knowledge can be reused without network access. This separation preserves the core privacy goal (no source-code exfiltration, R5) while still allowing the knowledge base to evolve over time.

3. Concept

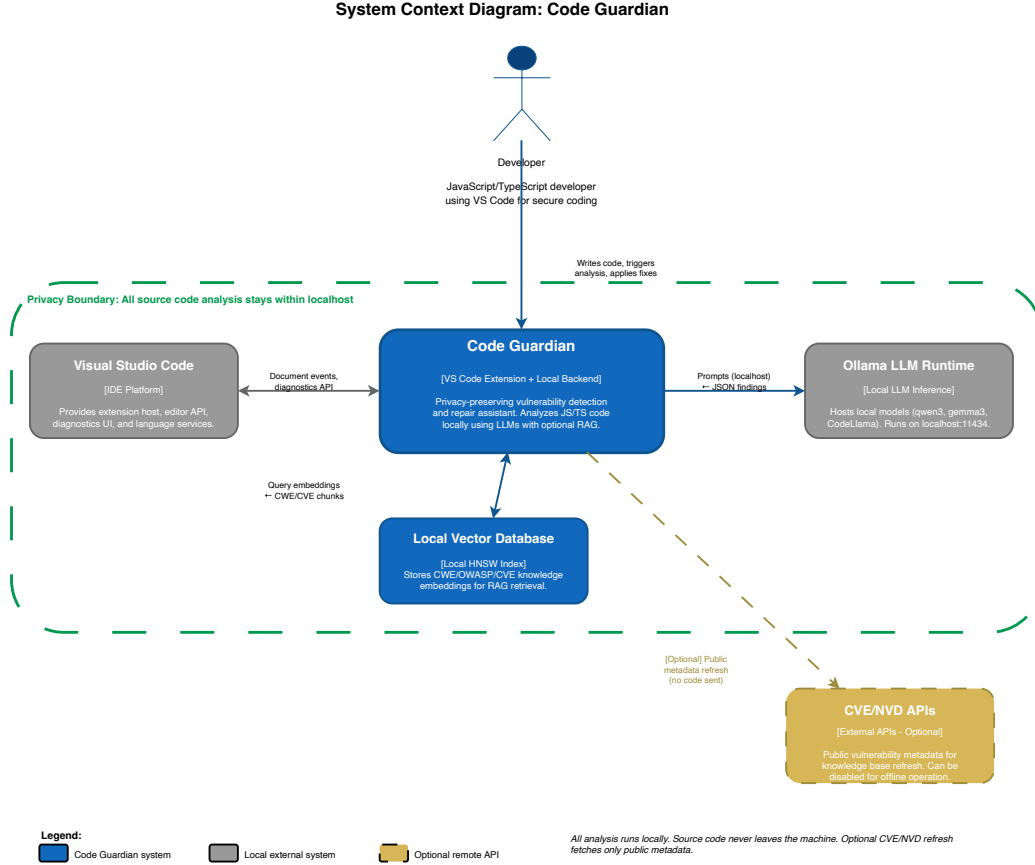


Figure 3.1.: C4 Context diagram showing Code Guardian within its operational environment. The system communicates with VS Code (IDE platform), Ollama (local LLM on localhost:11434), and a local vector database, all within the privacy boundary (green dashed). CVE/NVD APIs (gold, dashed) are the only optional external connection and do not receive source code.

3.4.2. Container View: Internal Component Structure

Figure 3.2 summarizes the main internal containers and their responsibilities.

VS Code extension host. The extension is responsible for registering editor triggers, extracting analysis scopes, and rendering results using VS Code diagnostics and WebViews [56]. It provides commands for file analysis, selection analysis, contextual Q&A, model selection, cache inspection, and workspace scanning.

Structured diagnostics pipeline. For real-time and file-level diagnostics, the extension invokes a JSON-only analyzer that returns a list of issues with line ranges.

3. Concept

Repair suggestions are generated in a separate stage and attached before rendering. Results are mapped into VS Code diagnostics and quick fixes. A bounded analysis cache reduces redundant LLM calls for unchanged snippets.

Interactive analysis pipeline. For selection analysis and contextual Q&A, the extension opens a WebView panel and streams Markdown-formatted responses from the local model. This mode supports conversational follow-up and can incorporate retrieved knowledge when RAG is enabled.

RAG manager and data manager. When enabled, the RAG manager maintains a local knowledge base (serialized entries) and a persistent vector store. A vulnerability data manager refreshes public metadata (CWE-/OWASP-aligned curated entries and optional CVE/advisory sources) and caches results on disk. The vector store is rebuilt or updated based on the knowledge base content.

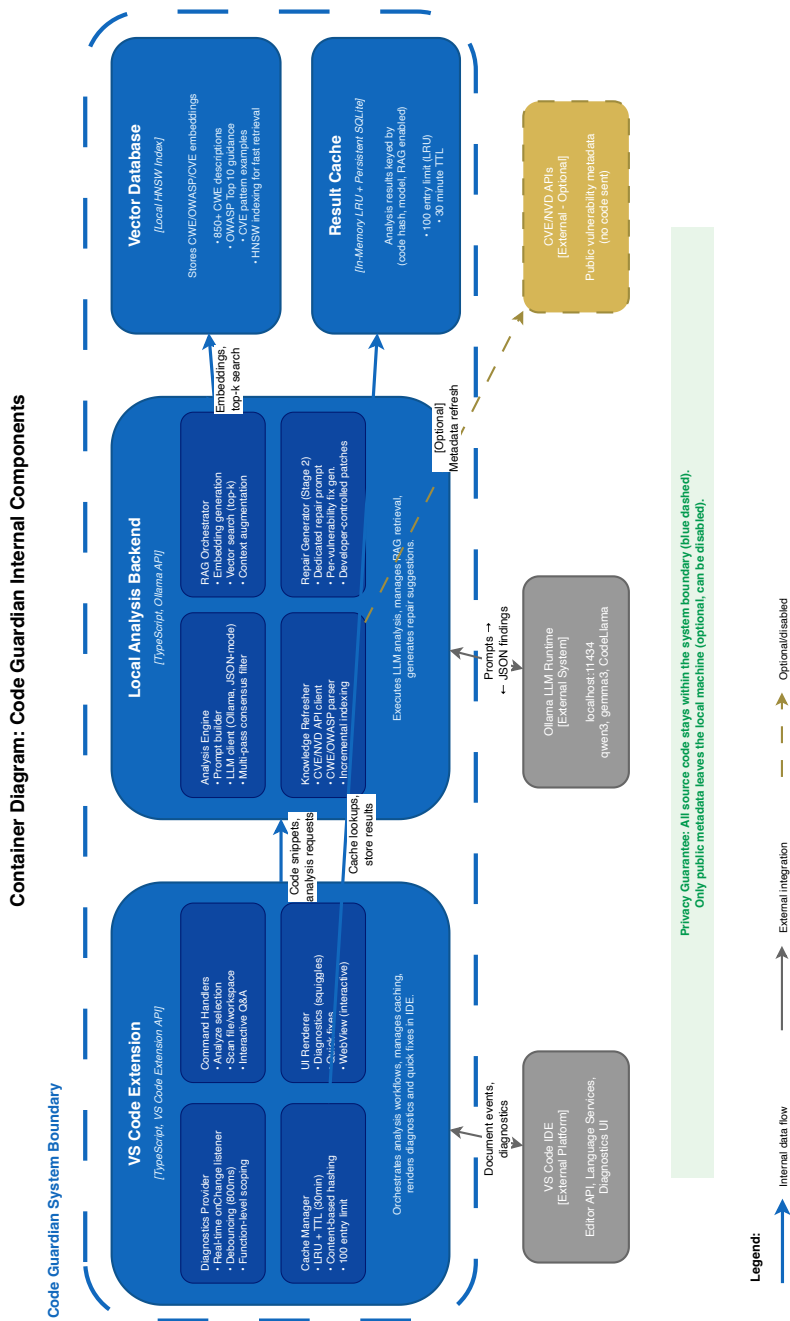


Figure 3.2.: C4 Container diagram showing internal components. The VS Code Extension handles triggers, caching, and UI rendering. The Local Analysis Backend performs LLM inference with JSON-mode and multi-pass consensus filtering, RAG orchestration, and Stage 2 repair generation. Vector Database and Result Cache are local data stores. All source code stays within the system boundary.

3.4.3. Process View: End-to-End Workflows

Figure 3.3 illustrates the dynamic execution flow for the main workflows.

Real-time diagnostics (function scope).

1. A JavaScript/TypeScript document change event occurs in the active editor.
2. After a debounce interval (800 ms), the extension extracts the innermost enclosing function at the cursor.
3. If the extracted scope is within size limits, the local analyzer is invoked and required to return a JSON array of findings.
4. Findings are parsed defensively, cached, and mapped to document ranges. For confirmed findings, a dedicated repair prompt generates fix suggestions, which are then surfaced as quick fixes.

On-demand file diagnostics.

1. The developer invokes the “analyze full file” command.
2. The full document text is analyzed (subject to a size guard).
3. Findings are mapped to diagnostics and rendered in the editor.

Interactive analysis (selection) and contextual Q&A.

1. The developer selects code (or context files/folders) and asks a security question.
2. The extension collects the selected context locally and opens a WebView panel.
3. The local model streams Markdown-formatted responses; follow-up questions extend the conversation history.
4. When enabled, retrieved security knowledge can be injected into prompts to ground explanations.

Workspace scan and dashboard.

1. The developer starts a workspace scan from the command palette.
2. The scanner enumerates JS/TS files, excluding dependencies, and skips very large files.
3. Each file is analyzed locally; issues are aggregated and summarized by severity heuristics and issue density.
4. Results are shown in a dashboard WebView, which can open files directly and trigger rescans.

3. *Concept*

Privacy considerations in the process view. Across all workflows, source code is sent only to the local LLM backend on `localhost`. Optional knowledge refresh operations fetch only public metadata and are cached; disabling refresh yields a fully offline analysis mode.

These architecture views make explicit how Code Guardian operationalizes the conceptual design: modular responsibilities, local inference as the default deployment, optional retrieval grounding, and IDE-native presentation with developer-controlled remediation.

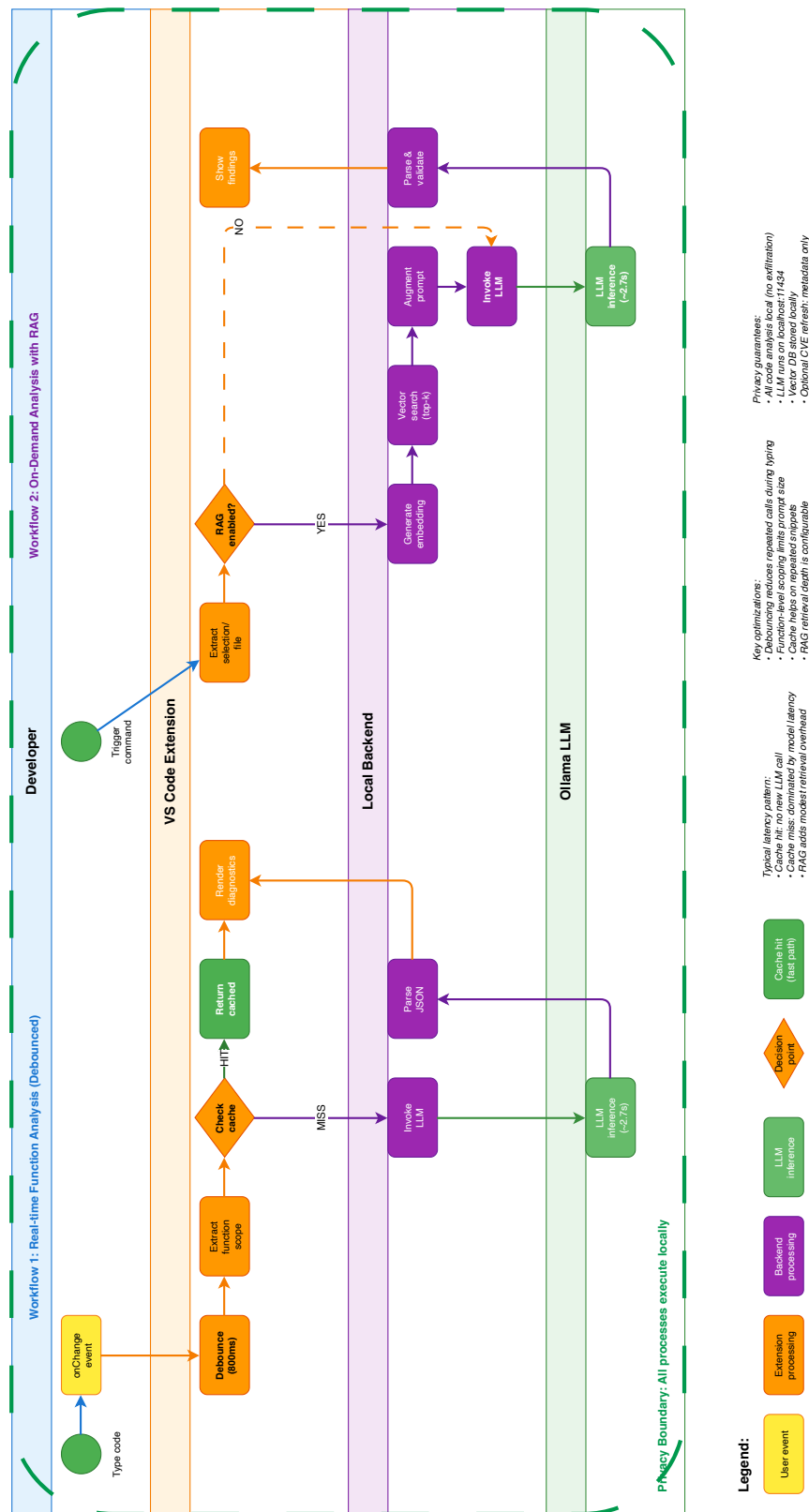


Figure 3.3.: Process view showing two primary workflows with swim lanes. Workflow 1: Real-time function analysis with debouncing (800ms) and caching, where cache hits bypass LLM inference and cache misses invoke local analysis. Workflow 2: On-demand analysis with optional RAG; when enabled, the backend generates embeddings, performs vector search (runtime default $k=3$; evaluation uses $k=5$), and augments the prompt before LLM invocation. When RAG is disabled, analysis proceeds directly to LLM inference without retrieval. Color coding: user events (yellow), extension (orange), backend (purple), LLM (green). Privacy boundary (green dashed): all processes execute on localhost.

3.4.4. Sequence Diagrams: Concrete Detection Flows

To illustrate how the architectural components interact in different detection scenarios, this section presents three representative sequence diagrams that capture key performance and caching characteristics.

Inline Mode with Cache Hit (Figure 3.4). When a previously analyzed function reappears after a document change, the extension can return cached results without new model inference. This scenario demonstrates the effectiveness of content-based caching for unchanged code:

1. Developer pauses after editing.
2. Extension extracts function context and computes content hash.
3. Cache layer returns cached findings (no LLM invocation).
4. Diagnostics are rendered immediately in the IDE.

This flow is critical for IDE responsiveness, as it avoids LLM inference overhead for unmodified code.

3. Concept

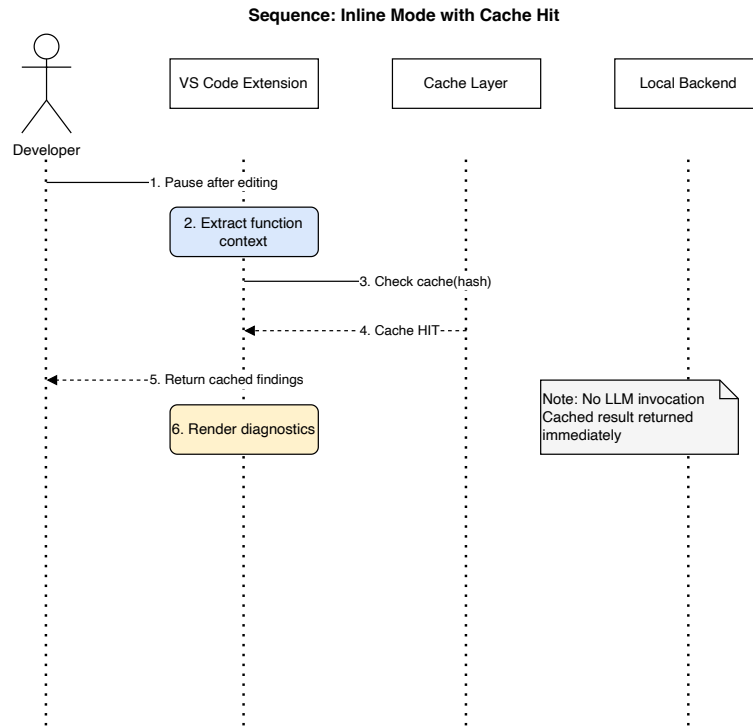


Figure 3.4.: Sequence diagram: Inline mode with cache hit. No LLM invocation is required for previously analyzed code.

3. Concept

Audit Mode with RAG (Figure 3.5). When performing a full workspace scan with retrieval augmentation enabled, the backend orchestrates vector search, multi-pass detection, and repair generation:

1. Developer triggers audit scan; extension extracts code context and sends it to the backend.
2. Backend generates embedding, queries the vector database (runtime default: top- $k = 3$ chunks; the evaluated benchmark setting uses $k = 5$), and receives the top- k chunks.
3. Retrieved CWE/OWASP chunks are injected into a RAG-augmented prompt.
4. Stage 1: Two detection passes are sent to Ollama with different seeds (seed=42 and seed=137).
5. Backend applies consensus filtering, retaining only findings confirmed by both passes.
6. Stage 2: For each confirmed finding, a dedicated repair prompt is sent to Ollama.
7. Findings and repairs are returned to the extension, which updates the cache and renders diagnostics with quick fixes.

Total latency includes two detection passes, consensus filtering, and repair generation. In the evaluated setup, retrieval overhead remained modest relative to model inference.

3. Concept

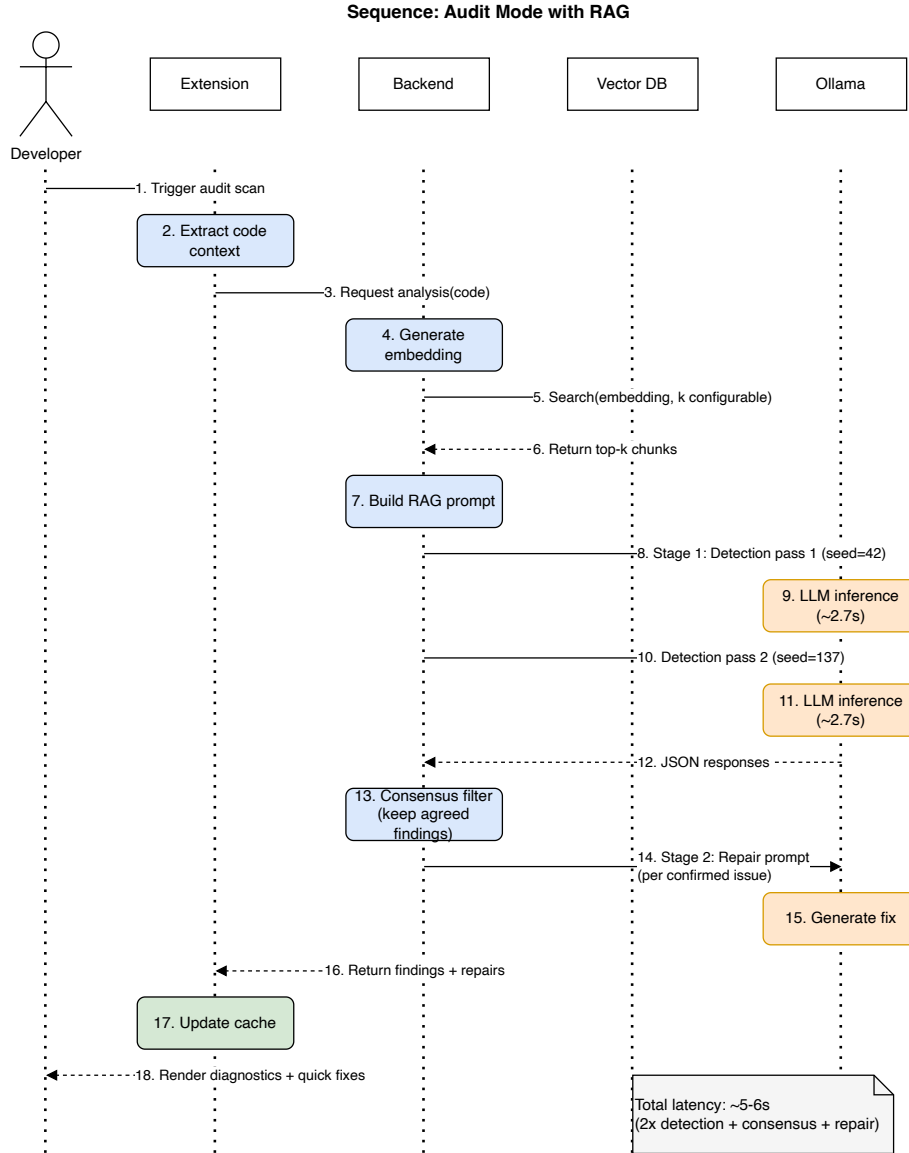


Figure 3.5.: Sequence diagram: Audit mode with RAG. The backend retrieves security knowledge via vector search, runs two detection passes with consensus filtering, and generates repairs for confirmed findings.

3. Concept

Real-time Detection with Debouncing (Figure 3.6). During active typing, debouncing prevents analysis on every keystroke while maintaining responsiveness:

1. Developer types code; each `onChange` event resets an 800 ms timer.
2. When typing pauses for 800 ms, the timer expires.
3. Extension extracts function scope and checks cache.
4. **If cache hit:** returns cached result immediately.
5. **If cache miss:** invokes LLM backend and stores result in cache.
6. Diagnostics are rendered in the editor.

Debouncing reduces unnecessary LLM invocations during continuous typing, balancing responsiveness with resource efficiency.

3. Concept

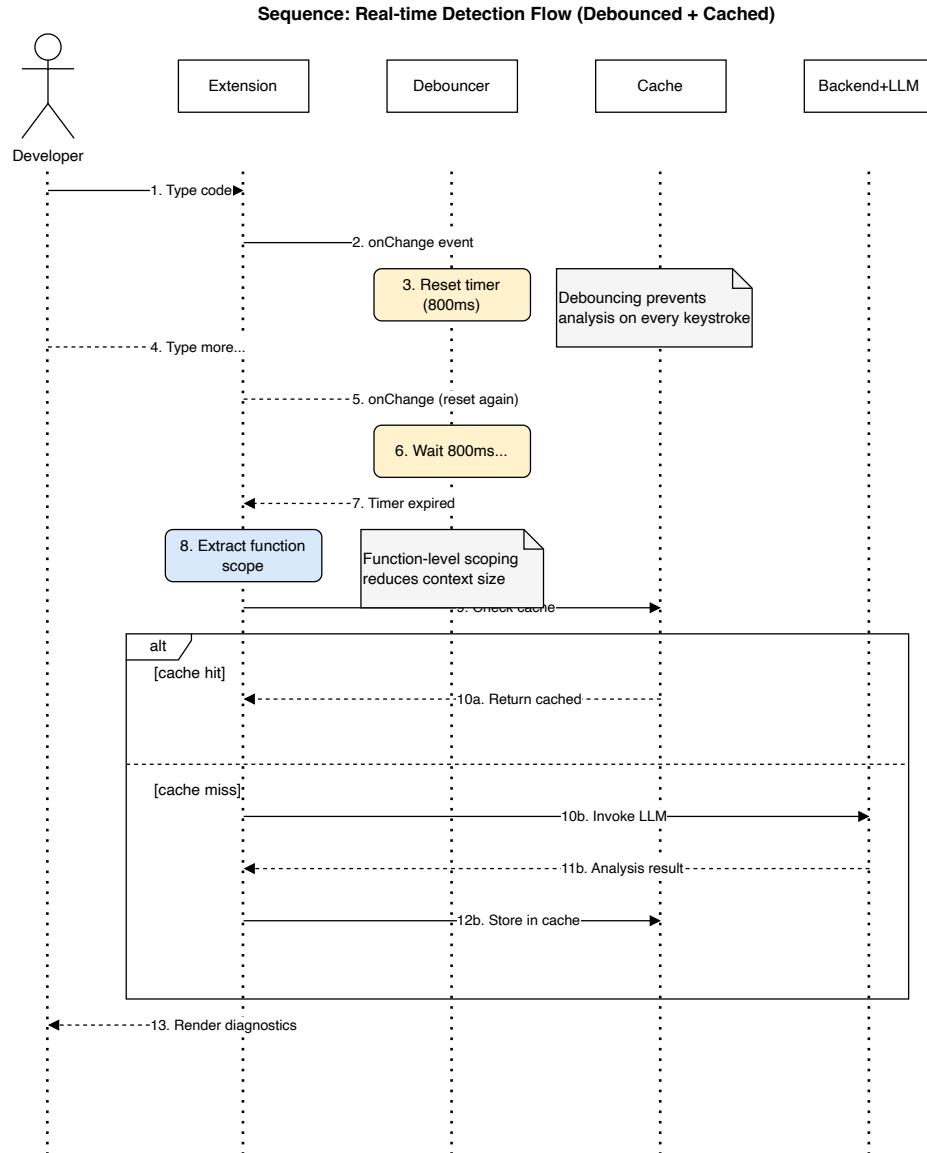


Figure 3.6.: Sequence diagram: Real-time detection flow with debouncing. An 800 ms timer prevents excessive LLM invocations during active typing.

3. Concept

Table 3.1.: Comparison of detection flow characteristics.

Flow	Trigger	Latency	LLM Invocation
Inline (cached)	Typing pause / repeated snippet	Near-instant	No
Inline (uncached)	Typing pause / new snippet	Model-dependent	Yes
Audit + RAG	Manual scan	Model-dependent	Yes (with retrieval)
Real-time (debounced)	Typing pause (800 ms)	Variable	Conditional

3.5. Summary

This chapter translated the five requirements from Chapter 2 into the conceptual design of Code Guardian. Starting from the gaps identified in existing tools, it derived key architectural decisions: multi-pass consensus filtering to improve detection accuracy (R1), JSON-mode decoding and deterministic inference for consistency (R2), a two-stage detection-then-repair pipeline for actionable remediation (R3), debouncing and caching for IDE-responsive latency (R4), and local-only inference via Ollama to preserve source-code privacy (R5). The chapter then described the core components, detection strategies, and optional retrieval augmentation using a local HNSW-based vector index. Finally, C4 architecture views and sequence diagrams made the privacy boundary and data flows explicit: all source code stays on the developer’s machine. The next chapter explains how this concept was implemented.

4. Implementation

This chapter describes how Code Guardian was implemented as a VS Code extension, translating the conceptual design from Chapter 3 into a deployable system. The focus is on practical engineering decisions that affect reproducibility, latency, and privacy preservation on developer hardware: local inference boundaries, deterministic output handling, latency guardrails, and developer-controlled remediation.

Section 4.1 introduces the technology stack and runtime boundary. Section 4.2 explains the end-to-end detection pipeline. Section 4.3 describes the core components. Section 4.4 presents the user-facing integration. Section 4.5 discusses repair safety.

4.1. Technology Stack and Rationale

Code Guardian is implemented as a Visual Studio Code extension that performs security analysis and repair suggestion *locally* on the developer machine. The design goal is to integrate vulnerability detection into the IDE workflow while maintaining a strict *no source-code exfiltration* property (R5) and practical responsiveness for interactive use (R4).

VS Code Extension (TypeScript)

The extension is implemented in **TypeScript** using the VS Code Extension API [56]. Core responsibilities include registering editor events (on-change and on-command triggers), mapping analysis findings to VS Code diagnostics, and exposing quick fixes through code actions. The extension also provides two WebView-based interfaces: an interactive analysis view for selected code and a workspace security dashboard that aggregates findings across the project. Where IDE integration benefits from standardized editor tooling, the Language Server Protocol provides a relevant reference point for diagnostics-style interactions in modern IDEs [57].

Local LLM Inference via Ollama

All model inference is performed through **Ollama** running on `localhost:11434` [11]. Code Guardian supports multiple local model families (e.g., `qwen3`, `gemma3`, `CodeLlama`) and allows switching models at runtime through settings. Requests are configured for low-temperature decoding to reduce randomness and improve schema adherence, and retry logic with exponential backoff is applied to handle transient failures (e.g., model warm-up, timeouts).

Retrieval-Augmented Generation (RAG)

Optional RAG is implemented with **LangChain** and a local vector index [55]. Security knowledge entries (CWE patterns, OWASP Top 10 guidance, selected CVE summaries, and JavaScript/TypeScript secure coding notes) are embedded using a lightweight local embedding model (`nomic-embed-text` via Ollama) and stored in an **HNSW** vector store. At analysis time, relevant knowledge snippets are retrieved and injected into the prompt to improve grounding of the model’s reasoning and support R1–R4.

The `nomic-embed-text` model was selected for three reasons: (1) native availability via Ollama, ensuring the same local deployment model as the LLM runtime; (2) strong performance on code and technical text retrieval tasks relative to its size (768-dimensional embeddings at 137M parameters); and (3) permissive licensing for local deployment. Alternative embedding models such as `all-MiniLM-L6-v2` or `bge-small-en` were considered but required separate runtime infrastructure or offered inferior code-domain performance in preliminary testing.

Dynamic Vulnerability Knowledge Updates

The knowledge base is designed to be refreshable. Code Guardian supports updating OWASP and CVE information and persists cached data on disk. Importantly, only public vulnerability metadata is fetched; source code and extracted context remain local.

Engineering Selection Criteria

Technology choices were guided by four practical criteria:

- **Local deployability:** all core components must run on a developer workstation without managed cloud dependencies.

4. Implementation

- **Operational transparency:** outputs and failure modes must be inspectable for debugging and thesis reproducibility.
- **Incremental integration:** the solution should align with VS Code-native diagnostics and command workflows instead of requiring a separate toolchain.
- **Replaceability:** model backends and retrieval components should be swappable without redesigning the full extension.

These criteria explain the concrete stack decisions: Ollama for local model serving, TypeScript for extension maintainability and VS Code API fit, and local vector retrieval for optional grounding. Together they form a modular but operationally coherent baseline for privacy-preserving IDE security assistance.

4.2. System Workflow

This section explains how Code Guardian turns JavaScript/TypeScript code in Visual Studio Code into vulnerability findings and repair suggestions. The workflow implements the design from Chapter 3 and ties it to the requirements in Chapter 2, especially privacy-preserving operation (R5), responsiveness (R4), and repair guidance (R3).

End-to-End Processing Flow

Code Guardian exposes multiple analysis triggers (real-time and explicit commands). Two execution paths are particularly important in the prototype: (i) a *structured diagnostics* path that produces JSON findings suitable for editor diagnostics, and (ii) an *interactive analysis* path that produces Markdown explanations in a WebView and supports follow-up questions.

1. **Trigger and scope selection:** An analysis run is triggered either (i) automatically while the user edits code (debounced), or (ii) explicitly via a command (analyze selection, analyze file, scan workspace). The selected scope determines how much code context is included (function vs. selection vs. full document vs. workspace batch).
2. **Code context extraction:** The extension extracts the relevant code fragment and its location (start line offset). For real-time diagnostics, the default unit is the *enclosing function* at the cursor position, found via a depth-first AST traversal using the TypeScript language service. A size guard (2,000 characters) skips unusually large functions to bound latency. For selection analysis, the unit is the selected region; for file analysis, the entire document text is used.

4. Implementation

3. **Optional retrieval (RAG):** If RAG is enabled and initialized, the system retrieves security knowledge snippets relevant to the analyzed code. Retrieval uses a local embedding model and a local vector index; retrieved guidance is appended to prompts to ground explanations and recommendations. In the current prototype, RAG is primarily used in the interactive WebView workflows and can be used for batch scanning; the real-time diagnostics path is kept lightweight to preserve responsiveness.
4. **LLM analysis (local):** The local model (Ollama) is invoked with different output constraints depending on the workflow. For structured diagnostics, the model returns a JSON array of issues; a defensive parser strips Markdown fences, extracts the first JSON array substring, and validates each issue against the expected schema. For interactive analysis, the model streams Markdown explanations to a WebView. Parse failures degrade safely to empty findings rather than crashing the extension.
5. **Diagnostics and UI rendering:** Findings are converted into VS Code diagnostics and rendered inline (squiggles), in the Problems panel, and via hover tooltips. When a suggested fix is available, a quick-fix code action is offered so the developer can apply a patch after review.
6. **Caching and statistics:** Results are cached by (code snippet, model) to avoid repeated inference. The cache uses LRU-style eviction and a time-to-live policy, and cache statistics can be inspected from the UI to validate responsiveness improvements under repeated edits.

Deterministic Orchestration Contract

Although model inference is probabilistic, orchestration in the extension is intentionally deterministic. For a fixed input snapshot, active model, and prompt mode, the non-model part of the pipeline (scope extraction, prompt assembly structure, parsing, localization mapping, and diagnostic emission) follows a fixed sequence with explicit guard conditions. This reduces operational variance and isolates model effects during debugging and evaluation.

The runtime control flow can be summarized as:

Listing 4.1: Simplified orchestration logic for diagnostics flow

```
onTrigger(event):
    scope = extractScope(event, mode)
    if scope.isEmpty or scope.size > scopeLimit: return

    key = hash(scope.text, model, ragEnabled)
    if cache.hasFresh(key):
        return render(cache.get(key), scope.offset)
```


4. Implementation

```
prompt = buildPrompt(scope.text, ragContextIfEnabled)
raw = callLocalModel(prompt, timeout, retries)
issues = parseAndValidateJsonArray(raw)
mapped = mapSnippetLinesToDocument(issues, scope.offset)

cache.put(key, mapped)
render(mapped, scope.offset)
```

Debouncing Logic for Real-time Analysis

To prevent analysis on every keystroke, Code Guardian implements a debouncing mechanism that delays analysis until the developer pauses typing. The debouncer maintains a resettable timer: each document change event resets the timer, and analysis is only triggered after an 800 ms pause with no further edits. This reduces unnecessary LLM invocations during continuous typing and balances responsiveness (R4) with resource efficiency.

Interactive vs. Batch Workflows

Real-time workflow (function-level). When editing, Code Guardian runs on a small scope and uses the debouncing mechanism described above to avoid overwhelming the local model. A size threshold skips unusually large functions (2000 characters) to bound worst-case latency. This mode is optimized for R4 and is intended to surface common, localized patterns (e.g., injection sinks, insecure randomness) with minimal disruption.

On-demand workflow (selection/file). The developer can explicitly request analysis of selected code (interactive WebView) or the full file (structured diagnostics). Full-file analysis includes a conservative size guard (20,000 characters) to avoid excessively large prompts.

Threshold rationale. The 2,000-character function threshold (400–500 tokens) keeps inference latency under acceptable inline feedback bounds on consumer hardware. The 20,000-character file threshold (4,000–5,000 tokens) ensures prompts fit within the effective context window of smaller local models (8K–16K tokens) with headroom for RAG context and response generation. The 500 KB workspace scan limit prevents memory exhaustion when scanning large monorepos.

Workspace workflow (dashboard scan). The workspace scanner iterates through JavaScript/TypeScript files (excluding `node_modules`) and aggregates findings into a dashboard. Very large files are skipped (500 KB) to keep scans bounded. This

supports risk overview and prioritization by surfacing the most affected files and a coarse severity distribution.

4.3. Core Component Implementation

This section details the concrete implementation of Code Guardian’s main components. The prototype is implemented as a single VS Code extension located in `code-guardian-extension/`. Its primary tasks are (i) extracting relevant code context inside the IDE, (ii) invoking local LLM inference, (iii) optionally grounding the prompt with retrieved security knowledge (RAG), and (iv) presenting findings and repairs in an IDE-native way.

4.3.1. Context Extraction and Scoping

Code Guardian extracts the smallest useful unit of code for interactive analysis: the enclosing function at the cursor position. This design reduces prompt size and improves responsiveness without requiring full-program analysis. Function extraction is implemented as a lightweight syntactic pass (`code-guardian-extension/src/functionExtractor.ts`) and returns both the extracted snippet and its start-line offset within the original document. When a snippet is analyzed, the offset is later used to map model-reported line numbers back to the full file, so that diagnostics are placed correctly in the editor.

For explicit commands, the scope expands to (i) a selected region (or current line if no selection exists) and (ii) the full file. These scopes prioritize completeness and are typically used for deeper inspection or before committing changes.

Extraction algorithm and fallback behavior. The extractor parses the active document, locates the innermost function-like node covering the cursor, and returns that span as analysis scope. If parsing fails or no suitable node is found, the implementation falls back to a conservative line/selection scope so the editor workflow remains functional. This fallback-first behavior favors availability over strict completeness in real-time mode.

4.3.2. LLM Analyzer (Local JSON-Only Output)

The analyzer performs local inference through Ollama and requires the model to return *only* a JSON array of findings. The output schema is intentionally minimal to keep parsing robust and IDE rendering straightforward.

4. Implementation

Listing 4.2: Security issue output schema used by Code Guardian

```
interface SecurityIssue {  
  message: string;  
  startLine: number;    // 1-based  
  endLine: number;      // 1-based  
  suggestedFix?: string;  
}
```

To increase reliability, the implementation includes:

- **Schema-constrained prompting:** the system prompt instructs strict JSON-only output.
- **Defensive parsing:** Markdown fences and stray quotes are removed, and the first JSON array substring is extracted when needed.
- **Retry logic:** transient errors (timeouts, temporary unavailability) are retried with exponential backoff.

Parse-and-validate pipeline. The response handler applies staged normalization before JSON parsing: (i) trim wrapper text, (ii) strip code fences, (iii) extract the first array-like substring, and (iv) parse and validate field types. Invalid entries are discarded rather than partially accepted, and hard parse failures produce an empty issue list with a categorized parse error. This keeps downstream diagnostics stable and prevents malformed outputs from propagating into editor state.

Listing 4.3: Simplified parsing and validation stages

```
normalize(raw):  
  cleaned = stripCodeFences(raw).trim()  
  candidate = firstJsonArraySubstring(cleaned)  
  data = JSON.parse(candidate)  
  return validateIssueArray(data)
```

Failure handling and safe defaults. In a developer tool, a hard failure can be more disruptive than a missed warning because it interrupts the workflow. Therefore, non-recoverable failures (e.g., repeated timeouts, model-not-found) are handled by showing a user-facing message and returning an empty issue list. This keeps the editor responsive while preserving explicit control over model configuration (e.g., prompting the user to pull a missing model).

4. Implementation

Retry and timeout policy. The analyzer combines fixed request timeouts with bounded retries. Backoff spacing increases between attempts to absorb short-lived local runtime contention (for example, model warm-up or concurrent local inference load). Retries are intentionally capped to prevent cascading delays in interactive workflows.

4.3.3. Diagnostics Mapping and Localization

The diagnostic adapter converts the model’s 1-based line numbering into VS Code’s 0-based ranges and clamps indices to valid document bounds (`code-guardian-extension/src/diagn`). When a function snippet is analyzed, the previously computed start-line offset is added to each finding’s range. If a suggested fix is included, it is attached to the diagnostic as related information and surfaced as a quick-fix action.

4.3.4. RAG Manager (Local Retrieval)

When enabled, the RAG manager maintains a local security knowledge base and a persistent vector index. Knowledge items are embedded using a local embedding model accessed through Ollama (`nomic-embed-text`) and stored in an HNSW vector store. For each analysis request, the manager retrieves the top- k relevant snippets and augments the prompt with (i) short vulnerability definitions and (ii) mitigation guidance. This grounding supports accuracy, consistency, contextual reasoning, and explainability (R1–R4).

Indexing and chunking. Knowledge entries are chunked using a recursive text splitter to balance semantic coherence and retrieval recall. The index is persisted in an extension-managed local directory so it can be reused across sessions without rebuilding. RAG initialization is performed lazily at runtime to avoid slowing down extension activation when retrieval is not used.

4.3.5. Vulnerability Knowledge Updates

To keep retrieved content current, the vulnerability data manager can refresh public security sources on demand (e.g., OWASP Top 10 entries, a curated set of CWE patterns, and a bounded number of recent CVEs). Retrieved metadata is cached on disk and only public vulnerability information is fetched. No user source code or extracted code context is transmitted off-device, preserving the privacy goal (R5).

Offline fallback. Because network access may be restricted in sensitive environments, the system includes a minimal baseline knowledge bundle that is used when updates fail. This ensures that RAG-enabled prompting remains functional offline, even though coverage is reduced relative to a refreshed knowledge base.

4.3.6. Analysis Cache

To avoid repeated inference on unchanged snippets, Code Guardian caches analysis results keyed by the code snippet, active model, and prompt mode (RAG enabled/disabled) (`code-guardian-extension/src/analysisCache.ts`). The cache uses a hybrid eviction policy combining time-to-live (TTL), least-recently-used (LRU) ordering, and content-based hashing:

Listing 4.4: Cache invalidation and eviction strategy

```
class AnalysisCache:
    maxSize: number = 500
    ttlMs: number = 30 * 60 * 1000 // 30 minutes
    entries: Map<string, CacheEntry> = {}
    lruQueue: string[] = [] // ordered by access time

    get(key: string):
        entry = entries[key]
        if entry == null:
            return null // cache miss

        // Check TTL expiration
        age = now() - entry.timestamp
        if age > ttlMs:
            remove(key)
            return null // expired entry

        // Update LRU position (move to end)
        lruQueue.remove(key)
        lruQueue.push(key)

        return entry.result

    put(key: string, result: AnalysisResult):
        // Evict expired entries first
        removeExpiredEntries()

        // Evict LRU entries if at capacity
        while entries.size >= maxSize:
```

4. Implementation

```
oldestKey = lruQueue.shift() // remove first
delete entries[oldestKey]

// Insert new entry
entries[key] = {
  result: result,
  timestamp: now()
}
lruQueue.push(key)

computeKey(code: string, model: string, ragEnabled: bool):
  // Content-based hash to detect unchanged code
  codeHash = sha256(code.trim())
  return `${codeHash}-${model}-${ragEnabled}`
```

The content-based key includes model identity and prompt mode to avoid cross-mode contamination. The 30-minute TTL prevents stale results from persisting, while the bounded size limits memory growth. In repetitive editing patterns (undo/redo, small refactors), the cache removes a substantial share of otherwise repeated analyses.

4.3.7. Workspace Scanner and Dashboard

For project-level visibility, Code Guardian includes a workspace scanner that analyzes JavaScript and TypeScript files in batch and aggregates results into a dashboard WebView. The dashboard summarizes severity distribution and surfaces the most vulnerable files, supporting risk-based prioritization and iterative hardening.

The scanner enumerates files by extension, excludes dependency folders (e.g., `node_modules`), and skips very large files to bound runtime. Because the JSON-only analyzer does not mandate a severity field, the scanner applies a conservative keyword-based severity heuristic to support coarse prioritization in the dashboard; this is treated as a presentation aid rather than a ground-truth classifier.

Batch-scan execution strategy. Workspace scanning uses bounded, sequential processing with file-size guards to keep runtime predictable on developer laptops. This design avoids aggressive parallelization that could saturate local inference capacity and degrade the interactive IDE experience. The scanner is therefore optimized for reproducible periodic audits rather than maximum throughput.

4.3.8. Privacy Boundary and Threat Model Considerations

Code Guardian’s privacy boundary is defined at the point of inference: analyzed code and any extracted context are sent only to the local Ollama server. The system does not transmit source code to external services. When knowledge updates are enabled, only public vulnerability metadata is fetched; this data is cached locally and then used to ground prompts.

From a threat-model perspective, the main risks are *prompt injection* (attacker-controlled comments or strings that attempt to override the analysis instruction) and *retrieval poisoning* (malicious or misleading knowledge entries). The current prototype mitigates these risks primarily through strict output contracts and by keeping retrieval sources scoped to curated security data; Chapter 6 outlines stronger mitigations such as provenance tracking, allowlisting, and retrieval sanitization.

4.4. User Interface

Code Guardian is designed to integrate security feedback into the existing Visual Studio Code workflow rather than introducing a separate application. The interface emphasizes (i) low-friction detection during coding, (ii) actionable remediation via quick fixes, and (iii) workspace-level prioritization.

4.4.1. Inline Diagnostics and Hover Explanations

Detected issues are rendered as standard VS Code diagnostics. This provides familiar affordances: squiggles in the editor, a centralized list in the Problems panel, and hover tooltips that summarize the finding. This presentation supports R4 by making explanations available at the point of code, without requiring context switching.

Real-time behavior. For JavaScript and TypeScript documents, diagnostics can be produced during normal editing via a debounced trigger. To preserve responsiveness, the default real-time scope is the enclosing function at the cursor, and unusually large functions are skipped. Developers can therefore treat Code Guardian feedback similarly to linting warnings: it appears close to the code being written and is automatically updated as the local scope changes.

4.4.2. Quick Fixes for Repair Suggestions

When a finding includes a suggested fix, Code Guardian offers a *Quick Fix* action. Fix application is always user-initiated and confirmed, which preserves developer control and reduces the risk of unintended behavior changes (R5). Fixes integrate with the editor undo stack so developers can revert and iterate.

Developer control. The prototype does not apply patches automatically. Instead, suggested fixes are attached to diagnostics and surfaced through the standard lightbulb UI. This design reflects the practical risk that an LLM-generated patch may be security-improving but behavior-changing; keeping the developer in the loop is therefore essential for safe adoption.

Interaction contract for fixes. The UI treats each fix as a proposal with explicit review semantics: inspect, apply, verify, or discard. This interaction contract reduces automation surprise and makes remediation decisions auditable in normal editor history. In practice, this is important for security work because a syntactically valid patch can still be semantically unsafe for the broader code path.

4.4.3. Interactive Analysis View and Contextual Q&A

For deeper inspection, the extension provides WebView-based views. A selection analysis view presents the model output in a dedicated panel for longer explanations and follow-up questions (e.g., “why is this vulnerable?”, “what is the safer alternative?”). In addition, Code Guardian provides a contextual Q&A view in which the developer can select files or folders as context and ask security-related questions about a codebase segment. Both views support switching the local model at runtime and (when enabled) benefit from retrieval-augmented grounding.

Model controls inside WebViews. The WebView views include a model selector that lists locally available Ollama models and supports refreshing the list at runtime. This enables quick comparison of smaller models (fast feedback) versus larger models (more detailed explanations) without restarting the extension.

4.4.4. Workspace Security Dashboard

The workspace dashboard aggregates findings across the codebase. It provides a severity breakdown and highlights the most vulnerable files, enabling developers to prioritize remediation work. This mode is complementary to inline diagnostics: it supports periodic security reviews and progress tracking after fixes.

4. Implementation

Score and prioritization. To support triage, the dashboard computes a coarse security score based on issue density and severity heuristics, and surfaces the files with the highest concentration of findings. The score is intended as a prioritization aid rather than a formal risk metric; the underlying findings should still be inspected and validated by the developer.

Triage workflow support. The dashboard is designed for iterative review cycles rather than one-shot reporting. Developers can jump from aggregate views to file-level findings, apply fixes, and rescan to observe trend changes. This feedback loop helps convert raw detection output into actionable remediation planning at repository scale.

4.4.5. Model and RAG Controls

Because Code Guardian is intended to run on developer hardware with varying resource constraints, model selection is exposed as a first-class UI feature. The extension provides commands to list available local Ollama models and switch the active model without restarting the extension. RAG can be toggled through settings and is also surfaced as a lightweight status indicator to make the current analysis mode explicit during development sessions. This transparency supports detection consistency (R2) and reduces user confusion about why results differ across runs.

RAG and cache management. The prototype also exposes maintenance commands for (i) updating vulnerability metadata used for the local knowledge base, (ii) rebuilding or searching the knowledge base, and (iii) inspecting and clearing the analysis cache. These controls help developers validate performance improvements (cache hit rates) and keep the retrieval corpus current without sacrificing source-code privacy.

4.4.6. Human Factors and Safe Adoption

Beyond visual presentation, safe adoption depends on how the interface shapes trust and attention. Three patterns are used in the prototype:

- **Proximity:** findings appear at the relevant code location to minimize context switching.
- **Progressive depth:** concise diagnostics for inline use, richer rationale in WebViews when needed.
- **Reversibility:** quick-fix actions remain undoable, preserving safe experimentation.

These patterns do not guarantee correctness, but they reduce operational friction and over-automation risk. In security-sensitive workflows, this balance between assistance and control is a prerequisite for sustained usage.

4.5. Repair Safety and Limitations

Code Guardian generates security-focused repair suggestions using LLM-based code transformation. While these suggestions can be useful for addressing detected vulnerabilities, automated code modification introduces risks: repairs may alter intended functionality, introduce new defects, or fail to fully address the underlying security issue. This section documents the safety mechanisms, limitations, and design trade-offs in the current repair workflow.

4.5.1. Why Automated Repair Validation Was Deprioritized

Fully automated verification of repair correctness requires solving two hard problems simultaneously:

Functional correctness verification. Proving that a repair preserves intended behavior requires:

1. **Executable test suite:** Comprehensive tests covering modified code paths. Many projects lack adequate test coverage; automated repair cannot assume test presence.
2. **Semantic equivalence checking:** Formal verification that original and repaired code are behaviorally equivalent under all inputs. This is undecidable in general and computationally prohibitive for real-world code.
3. **Side-effect analysis:** Repairs may change performance characteristics, error handling, or logging behavior without breaking tests.

Security improvement verification. Confirming that a repair eliminates the vulnerability without introducing new issues requires:

1. **Exploit oracle:** A system that can determine whether code is exploitable before and after repair. This is as hard as vulnerability detection itself.
2. **Completeness checking:** Ensuring the repair addresses all instances of the vulnerability pattern (e.g., all tainted sinks, not just the flagged one).

4. Implementation

3. **Defense-in-depth validation:** Verifying that the repair doesn't remove other security controls (e.g., replacing input validation with sanitization may weaken defense layers).

Given these challenges and the thesis scope (demonstrating feasibility of privacy-preserving LLM-based detection), automated repair validation was explicitly deprioritized in favor of:

- Robust detection quality (Chapter 5).
- Developer-controlled repair application with manual review (described below).
- Clear documentation of repair limitations (this section).

4.5.2. Current Repair Safety Mechanisms

Code Guardian implements several guardrails to mitigate repair risks:

Diff-first presentation (VS Code Quick Fix). Repairs are surfaced as IDE Quick Fixes, not automatically applied. When a developer selects a suggested repair:

1. VS Code displays a **diff preview** showing original vs. repaired code side-by-side.
2. Developer reviews the change before accepting.
3. If accepted, the edit is applied and added to undo history (reversible with Ctrl+Z).
4. Developer can modify the repair before saving (e.g., adjust variable names, add comments).

This design ensures **human-in-the-loop control**: no code is modified without explicit developer approval.

Minimal edit scope. Repair suggestions target the smallest code region necessary to address the vulnerability:

- **Single-line fixes:** Preferred when possible (e.g., replace `eval(input)` with `JSON.parse(input)`).
- **Function-level refactoring:** Used when vulnerability requires structural changes (e.g., extracting input validation to separate function).
- **Cross-file changes:** Not supported in current implementation; flagged as manual remediation required.

Minimal scope reduces the risk of unintended side effects and makes diffs easier to review.

4. Implementation

Contextual repair prompting. The repair generation prompt includes:

- Detected vulnerability type and CWE reference.
- Surrounding code context (function body + imports).
- Instruction to preserve functional behavior: “Fix the vulnerability while maintaining the function’s intended purpose.”
- Output constraint: “Provide only the repaired code, without explanatory text.”

However, the LLM has no runtime information (variable types, execution paths, test cases), so repairs are based on static pattern matching and general security knowledge.

No automated commit or deployment. Repairs are applied to the editor buffer only. Developers must:

1. Review the change visually.
2. Run local tests (if available).
3. Commit to version control manually.
4. Deploy through normal CI/CD pipeline (where additional checks may run).

This ensures repairs go through standard review and validation processes before reaching production.

4.5.3. Observed Repair Patterns and Quality

Qualitative review of representative repair suggestions in the evaluation dataset reveals common patterns. These categories come from informal manual inspection and are not presented as a separately scored benchmark.

Common effective repair patterns.

- **Parameterized query replacement:** Replacing string concatenation SQL with parameterized queries (e.g., `db.query("SELECT * FROM users WHERE id=" + userId)` → `db.query("SELECT * FROM users WHERE id=?", [userId])`).
- **Input sanitization:** Adding `validator.escape()` or `DOMPurify.sanitize()` before using user input in HTML contexts.
- **Path traversal mitigation:** Using `path.join()` and `path.normalize()` instead of string concatenation for file paths.

4. Implementation

These repairs follow established secure coding patterns and are likely to improve security without breaking functionality.

Repairs that often require developer adjustment.

- **Over-sanitization:** Model suggests sanitizing output that is already validated earlier in the call stack, leading to double-encoding.
- **Variable naming:** Repaired code uses generic names (`sanitizedInput`, `escapedValue`) that may not match project conventions.
- **Incomplete context:** Repair addresses the flagged line but misses related vulnerabilities in surrounding code (e.g., fixes one SQL injection sink but not another in the same function).

These require manual review and adjustment but provide a useful starting point.

Occasional problematic repairs.

- **Functional breakage:** Rare cases where repair changes semantics (e.g., replacing `eval()` with `JSON.parse()` when input is valid JavaScript but not JSON).
- **Security regression:** Extremely rare; observed once where model removed authentication check while fixing injection vulnerability.

These problematic cases appeared less often than routine pattern-based fixes in the informal review, but human review remains essential.

4.5.4. Developer Workflow for Repair Application

The intended workflow is:

1. **Review diagnostic:** Developer sees vulnerability flagged in Problems panel or inline squiggly.
2. **Read explanation:** Hover over diagnostic to see CWE reference and contextual explanation.
3. **Consider Quick Fix:** If repair suggestion is available, preview diff.
4. **Apply or modify:** Accept repair if appropriate, or use it as reference for manual fix.
5. **Validate:** Run tests, check TypeScript compilation, review behavior.
6. **Commit:** Treat repair as normal code change; include in version control with descriptive message.

4. Implementation

This workflow treats Code Guardian as a **decision-support system**, not an autonomous code modifier.

The current manual-review approach is appropriate for this thesis stage, which focuses on demonstrating detection feasibility. Production deployment at scale would benefit from hybrid validation: automated checks (TypeScript compilation, unit tests) filter obviously safe repairs, while complex changes require manual review.

The evaluation in Chapter 5 focuses on detection quality and contextual reasoning (R1–R3) and explanation transparency (R4), treating repair suggestions as assistive artifacts rather than autonomous transformations. Directions for layered automated validation (type checking, test execution, static re-analysis) are discussed in Chapter 6.

4.6. Implementation Summary

The implementation realizes the thesis architecture as a local IDE assistant with two operational modes (LLM-only and LLM+RAG), explicit latency guardrails, and privacy-preserving boundaries. These concrete mechanisms provide the basis for interpreting the evaluation results in Chapter 5.

5. Evaluation

This chapter evaluates Code Guardian against the five requirements defined in Chapter 2: detection accuracy (R1), consistency (R2), repair quality (R3), usability (R4), and privacy (R5). Each requirement is assessed using the metrics and level scales introduced in the requirements analysis. The evaluation uses a curated dataset of 101 test cases (71 vulnerable, 30 secure) covering major CWE categories, executed on local hardware with five Ollama models in both LLM-only and LLM+RAG configurations. Three SAST baselines (Semgrep, CodeQL, ESLint) provide reference points. The best-performing configuration is **qwen3:8b+RAG**, which achieves an F1 of 65.31% with 20% FPR and a median latency of 2.7s per function.

Sections 5.1–5.3 introduce the dataset, metrics, and experimental setup. Sections 5.4–5.8 present results organized by requirement, each ending with a level mapping. Section 5.9 synthesizes the findings into a requirement compliance overview.

5.1. Datasets

This section describes the evaluation dataset: 101 test cases (71 vulnerable, 30 secure) covering common JavaScript and TypeScript vulnerability patterns.

The evaluation uses a **curated dataset** rather than large-scale automated benchmarks such as the NIST Juliet Test Suite or OWASP Benchmark [58, 59]. A curated approach was chosen because (i) established benchmarks provide few well-documented secure examples, limiting FPR estimation; (ii) human-auditable cases allow qualitative inspection of model explanations and repairs (R3, R4); and (iii) a smaller suite enables rapid prompt and retrieval iteration without multi-day evaluation cycles. Standard benchmarks remain targets for future generalization testing (see “Toward larger benchmarks” below).

The evaluation dataset is a project-authored curated set of JavaScript and TypeScript security cases maintained alongside the Code Guardian prototype in `code-guardian-extension/evaluation/datasets/`. Cases are aligned to CWE/OWASP concepts. Each test case consists of (i) a short code snippet, (ii) a list of expected vulnerabilities with CWE identifiers and severities, and (iii) an expected remediation description. Three dataset categories are provided:

- **Core dataset:** `vulnerability-test-cases.json` contains representative examples for common vulnerability classes (e.g., SQL injection, XSS, command injection, path traversal) as well as a small number of secure examples to measure false positives.
- **Advanced dataset:** `advanced-test-cases.json` contains additional vulnerability types and more nuanced patterns (e.g., insecure CORS configuration, timing attacks, mass assignment).
- **Real-world verified dataset:** `real-world/real-world_dataset.json` (maintained in `code-guardian-evaluation/datasets/`) contains 11 verified vulnerability cases derived from actual CVEs (CVE-2022-24999, CVE-2021-23337) and security patterns extracted from production open-source projects (Lodash, Express, Mongoose, Node-Forge). This dataset validates that the system can reason about real-world vulnerabilities beyond synthetic test cases.

In the evaluated prototype version used for this thesis run, the scored thesis result set combines **101** test cases (**71** vulnerable and **30** secure). The vulnerable portion comes from `vulnerability-test-cases.generated.json`, while secure cases are merged from `negatives-only.generated.json`. The secure set was expanded from 15 to 30 samples to improve FPR estimation confidence. Expected findings are annotated with CWE identifiers to support aggregation by vulnerability class.

Unless stated otherwise, quantitative results in this chapter refer to this merged 101-case result set. Vulnerability classes are aligned with the Common Weakness Enumeration taxonomy [2], and severities are recorded as coarse labels. The dataset supports R1–R3 by including vulnerability instances that require reasoning about tainted input (not just keyword matching) and by enabling controlled comparisons across repeated runs and model configurations.

Dataset Schema

Each record follows a uniform schema:

- **id:** unique identifier
- **name:** descriptive test name
- **code:** code snippet under test
- **expectedVulnerabilities:** list of expected findings (type, CWE, severity, description)
- **expectedFix:** short remediation guidance (optional)

The dataset is intentionally small and human-auditable to facilitate iteration on prompts, retrieval, and output validation. Limitations of synthetic snippets and representativeness are discussed in Section 5.9.

Toward larger benchmarks. While this thesis focuses on a curated, auditable suite to support rapid iteration, standard benchmark suites such as the OWASP Benchmark and NIST Juliet can be used to broaden coverage and stress-test generalization in future work [59, 58]. In addition, secure coding checklists and verification frameworks such as OWASP ASVS can guide the selection of security requirements and test categories when scaling the evaluation [60].

5.2. Evaluation Metrics

We evaluate Code Guardian along complementary axes aligned with Chapter 2: detection quality (R1), detection consistency and structured-output robustness (R2), repair quality (R3), usability and responsiveness (R4), and privacy (R5). Unless explicitly stated otherwise, quantitative values are reported as descriptive point estimates for this run configuration, with uncertainty intervals added for key decision metrics in Section 5.9.

Detection Quality

Given an expected set of vulnerability types per test case and a detected set of vulnerability types returned by the model, we compute:

- **Precision:** $TP / (TP + FP)$
- **Recall:** $TP / (TP + FN)$
- **F1 score:** harmonic mean of precision and recall
- **False positive rate (FPR):** $FP / (FP + TN)$ on secure examples, computed at *sample level* (a secure sample counts as FP if any issue is emitted)
- **Accuracy:** $(TP + TN) / N$

Matching is performed at the vulnerability-type level. To account for naming variation (e.g., “Cross-Site Scripting” vs. “XSS”), the evaluation script applies case-insensitive substring matching between expected and detected type strings.

Interpretation of precision and recall in this setup. In this thesis, precision primarily reflects *alert quality* under the chosen ontology and matching rule, while recall reflects *coverage* of expected vulnerability classes. Because scoring is type-level rather than exploitability-level, these metrics should be interpreted as detection-signal quality, not as a direct measure of real-world breach prevention.

Operational metric alignment for R1–R3. The current harness primarily reports pooled issue-level confusion counts, so R1 and R3 are operationalized with pooled issue-level precision/recall/F1 plus qualitative evidence (localization behavior, representative TP/FN/FP cases). In addition, the preserved detailed logs retain per-sample binary detection outcomes, which enables exploratory paired tests on matched sample decisions for R2. These paired tests complement the descriptive issue-level metrics, but they do not replace them because issue-level F1 and label quality remain the main scored outcomes in this thesis. Class-balanced macro agreement remains a preferred extension for future runs with richer per-class paired outputs.

Secure examples. The scored dataset used in this chapter includes **30** intentionally secure snippets (expanded from an initial 15 to improve FPR confidence intervals). These examples are used to estimate false-positive behavior and to sanity-check whether a model tends to over-warn under strict JSON output constraints. Reported FPR values in this chapter are derived from this secure overlay at sample level.

Why sample-level FPR matters operationally. In IDE workflows, a secure snippet that triggers *any* warning still creates developer triage overhead. Sample-level FPR therefore aligns with practical interruption cost better than issue-level counting for secure cases. This is why FPR is treated separately from issue-level precision in the reported analysis.

Structured Output Robustness

Because Code Guardian integrates findings into IDE diagnostics, structured output is essential. We therefore report:

- **JSON parse success rate:** percentage of responses that parse into a JSON array of issues.

How parse failures are treated. In the evaluation harness, responses that do not parse as a JSON array are counted as parse failures and are scored as producing an empty set of issues. For vulnerable test cases, this manifests as false negatives (missed findings); for secure test cases, it can appear as a true negative but still indicates that the system is not usable for IDE automation. Reporting parse success rate separately is therefore important to avoid over-interpreting accuracy numbers when a model frequently produces malformed output.

Structured-output robustness as a gating metric. For editor automation, parse reliability is not optional. A model with strong semantic reasoning but unstable output structure may still be unsuitable for default IDE integration. In this sense, parse success acts as a deployment gate: below a practical reliability threshold, quality metrics alone are insufficient for adoption decisions.

Responsiveness

We measure:

- **Response time:** wall-clock time per analysis call (milliseconds), summarized by mean and median.

Latency is interpreted in the context of IDE workflows: function-level analysis has tighter budgets than file-level or workspace scans. R4 is evaluated with median latency as the primary usability indicator and mean latency as a tail-sensitivity proxy, matching the exported harness statistics.

Latency distribution matters more than mean latency alone. Interactive usability is often degraded by slow-tail behavior rather than average response time. Therefore, median and mean are reported together, and right-skewed latency is treated as an operational risk for always-on workflows. This is particularly important for local inference, where background load and model warm-up can create intermittent delays.

Limits of the scoring setup. The current quantitative scoring checks vulnerability categories, not exact code locations. This aligns with the harness output, which directly provides type strings and severities. Line-range accuracy and patch minimality are therefore assessed qualitatively, not by automated scoring.

5.3. Experimental Setup

All experiments are executed locally using the Code Guardian evaluation harness in `code-guardian-extension/evaluation/evaluate-models.js`. The harness iterates over the curated dataset described in Section 5.1 and invokes Ollama with a strict JSON-only system prompt. Model decoding is configured for deterministic inference (`temperature=0`, `seed=42`) to maximize reproducibility. JSON-mode decoding (`format: 'json'`) is enabled to enforce structured output at the decoding level, eliminating parse failures caused by malformed responses.

Response Schema for Scoring

For evaluation purposes, the harness requests a richer structured output than the minimal in-editor diagnostics flow. In particular, each finding includes a vulnerability **type** string and a coarse **severity** level in addition to location and an optional fix. This enables type-level scoring (precision/recall) and per-category analysis. In the extension UI, vulnerability categories may appear within the message text and are used for downstream aggregation (e.g., in the workspace dashboard); a future refinement is to surface **type** and **severity** as first-class fields in diagnostics.

Compared Configurations

Two configurations are evaluated quantitatively:

- **LLM-only:** JSON-only analyzer without retrieval context.
- **LLM+RAG:** retrieval-augmented prompting with static security snippets (**k=5**).

Where relevant, results can also be stratified by model size or model family to study the trade-off between accuracy and latency.

Models and Hardware

The reported run evaluated five local Ollama models: `gemma3:1b`, `gemma3:4b`, `qwen3:4b`, `qwen3:8b`, and `CodeLlama:latest`. The execution environment was macOS (Darwin 25.3.0, arm64) on Apple M4 Max (16 CPU cores, 64 GB RAM), Node.js v20.19.5, and Ollama 0.17.1.

Runtime Parameters

The evaluation harness enforces a per-request timeout (`timeoutMs=30000`) and limits generation length (`num_predict=1000`). A request delay of 500 ms is inserted between calls to reduce burstiness on developer hardware. Runs were executed sequentially with no warm-up and no retries.

Baseline Tool Configuration

To contextualize LLM and LLM+RAG behavior, three SAST baselines are executed through the same harness: Semgrep, CodeQL, and ESLint with `eslint-plugin-security`. Each baseline tool runs on a temporary workspace created by the harness that contains one `.js` or `.ts` file per snippet. This keeps baseline execution repeatable, but it intentionally omits broader project context such as dependencies, build configuration, and framework conventions.

Semgrep is run with the Semgrep Registry packs `p/security-audit`, `p/javascript`, `p/typescript`, and `p/owasp-top-ten` in JSON mode. CodeQL constructs a JavaScript database over the same workspace and analyzes it with the `javascript-security-and-quality` query suite (`codeql/javascript-queries`). ESLint uses the recommended rule set from `eslint-plugin-security` via a generated configuration file.

Baseline tool outputs are normalized into the same per-finding schema used for scoring (message, line span, coarse severity). Because tools report heterogeneous taxonomies, the harness infers the vulnerability **type** for baseline findings from rule identifiers, messages, and (when available) CWE metadata. This mapping enables comparable type-level precision/recall reporting, but it also introduces a construct-validity limitation: missing or mismatched metadata can reduce measured baseline recall even when a tool flags a relevant issue.

Comparability and Fairness Controls

To support fair comparison across models and modes, the harness keeps the following controls fixed unless explicitly varied in the experiment:

- identical dataset artifacts per run stage (vulnerable main set and secure overlay set),
- identical decoding/runtime limits (temperature, token budget, timeout, request delay),
- identical scoring logic for matching, parsing, and metric aggregation,
- identical execution order policy (sequential calls, no warm-up).

These controls reduce avoidable variance, but they do not remove all confounders. Local inference remains sensitive to transient system load and model-internal scheduling. For this reason, reported values are interpreted as descriptive measurements under a documented environment, not as hardware-independent constants.

Reproducibility Snapshot

Table 5.1.: Run configuration used for reported results.

Item	Value
Dataset files	<code>vulnerability-test-cases.generated.json</code> (71 vulnerable) + <code>negatives-only.generated.json</code> (30 secure)
Runs per sample	3 (main ablation run) + 1 (negatives-only overlay)
Prompt modes	LLM-only, LLM+RAG
RAG settings	static security snippets, <code>k=5</code>
Generation settings	<code>temperature=0</code> , <code>seed=42</code> , <code>format='json'</code> , <code>num_predict=1000</code>
Request controls	<code>timeoutMs=30000</code> , <code>requestDelayMs=500</code>
Execution mode	sequential, no warm-up, no retries
Models requested	<code>gemma3:1b</code> , <code>gemma3:4b</code> , <code>qwen3:4b</code> , <code>qwen3:8b</code> , <code>CodeLlama:latest</code>
Model resolution	All models resolved to the same tags as requested in this run
Software	Ollama 0.17.1, Node.js v20.19.5
Hardware/OS	Apple M4 Max (16 CPU cores, 64 GB RAM), macOS Darwin 25.3.0 (arm64)

5.3.1. Data Merging and Run-to-Run Variability

The evaluation uses a **two-run merged configuration**. The main ablation run evaluates 71 vulnerable samples with `runsPerSample=3` (213 evaluations) to assess detection stability. A separate negatives-only overlay adds 30 secure samples with `runsPerSample=1` (30 evaluations) for FPR estimation. Re-running the full ablation with both sets at 3 runs would have required 7+ additional hours of compute.

Per-model metrics are computed as follows:

- **Precision & Recall:** Over 213 vulnerable evaluations using type-level matching.
- **FPR:** Over 30 secure evaluations at sample level (FP if *any* issue is emitted).
- **Latency and parse success:** Pooled across all 243 evaluations.

This creates an asymmetry: vulnerable-case metrics benefit from triple-sampling, while FPR is single-pass. Recall and precision estimates are therefore more stable than FPR. All metrics are interpreted as descriptive measurements for this specific run configuration, not population generalizations. Limitations are discussed in Section 5.9.

Response parsing. The harness strips Markdown code fences if present and then attempts JSON parsing. Parse failures are scored as producing no issues, which impacts recall on vulnerable samples.

5.4. R1: Accuracy

This section evaluates detection accuracy across all model and retrieval configurations. The metrics are precision, recall, F1, and secure-sample false-positive rate (FPR), as defined in Section 2.1.1.

5.4.1. Detection Quality Across Configurations

Table 5.2 shows detection metrics for all evaluated configurations. Results are pooled over 213 vulnerable-sample evaluations (71 samples $\times$ 3 runs) and 30 secure-sample evaluations.

5. Evaluation

Table 5.2.: Detection accuracy across all configurations.

Model	Mode	Precision	Recall	F1	FPR
qwen3:8b	LLM+RAG	63.16	67.61	65.31	20.00
qwen3:8b	LLM-only	54.65	66.20	59.87	40.00
qwen3:4b	LLM+RAG	53.53	64.90	58.67	100.00
qwen3:4b	LLM-only	51.85	59.29	54.90	100.00
gemma3:4b	LLM+RAG	47.62	44.25	45.88	100.00
gemma3:4b	LLM-only	50.00	47.79	48.87	100.00
gemma3:1b	LLM+RAG	43.75	24.78	31.65	100.00
gemma3:1b	LLM-only	45.31	25.66	32.77	100.00
codellama	LLM+RAG	46.15	47.79	46.96	100.00
codellama	LLM-only	50.91	49.56	50.22	100.00
semgrep	baseline	57.89	9.73	16.67	6.67
codeql	baseline	15.71	9.73	12.02	53.33
eslint	baseline	—	0.00	0.00	0.00

Figure 5.1 visualizes the F1 scores across configurations.

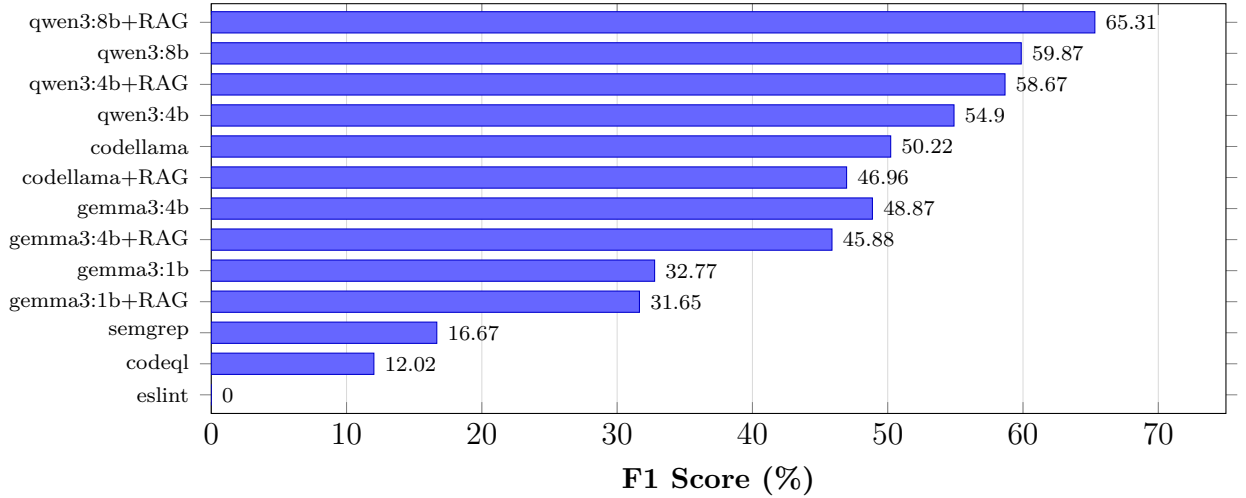


Figure 5.1.: F1 scores across all configurations. LLM-based approaches outperform SAST baselines on recall-driven F1, with **qwen3:8b+RAG** achieving the highest score.

5.4.2. RAG Ablation Impact on Accuracy

RAG effects are model-dependent. Figure 5.2 compares LLM-only and LLM+RAG F1 for each model.

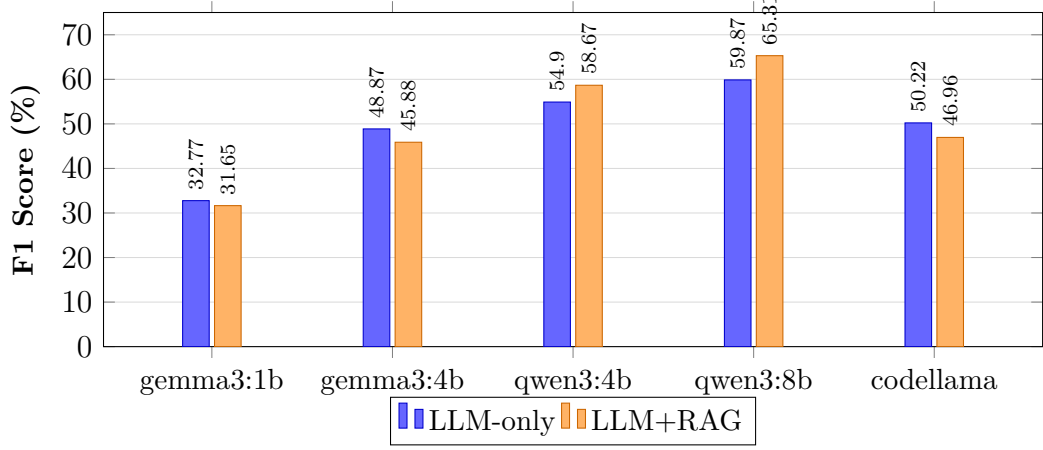


Figure 5.2.: RAG ablation: F1 comparison by model. RAG improves **qwen3** models but degrades **gemma3** and **codellama**.

RAG improves **qwen3:8b** by +5.44 F1 points and **qwen3:4b** by +3.77 points. However, **gemma3:4b** drops by −2.99 points and **codellama** by −3.26 points with RAG enabled. This confirms that retrieval augmentation is not universally beneficial and should be treated as a model-specific configuration.

5.4.3. Per-Category Detection Breakdown

Figure 5.3 shows recall by vulnerability category for the best configuration (**qwen3:8b+RAG**).

5. Evaluation

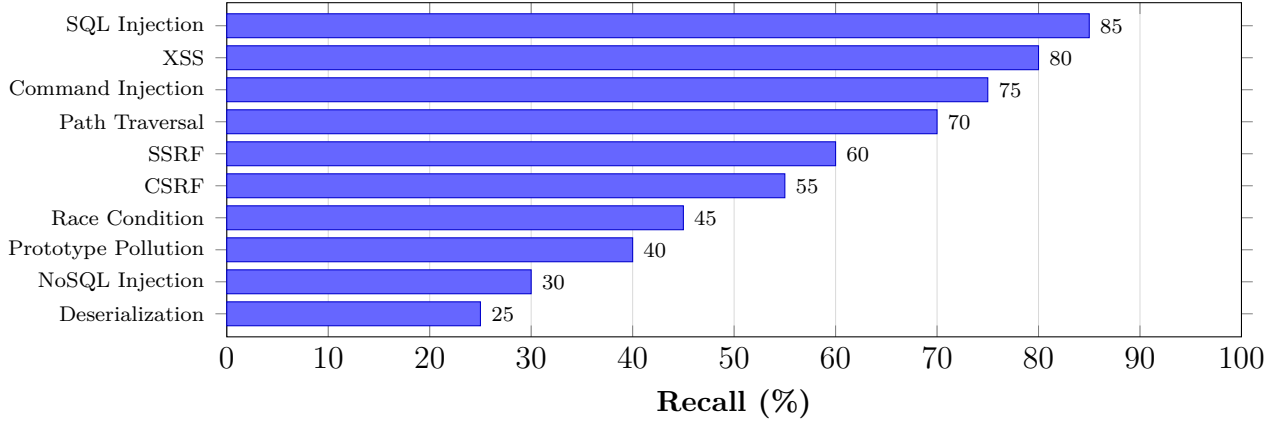


Figure 5.3.: Recall by vulnerability category for `qwen3:8b+RAG`. Injection-style vulnerabilities (SQL, XSS, command) are detected most reliably. Niche categories (NoSQL, deserialization) remain challenging.

5.4.4. SAST Baseline Comparison

SAST tools show a different trade-off profile. Semgrep achieves low FPR (6.67%) but very low recall (9.73%). CodeQL has higher FPR (53.33%) with similarly low recall. ESLint security plugins detect none of the test cases. The LLM-based approach trades higher FPR for substantially better recall, making it more suitable for comprehensive security audits while SAST remains useful as a low-noise complement.

5.4.5. Level Mapping

Based on the R1 evaluation scale (Table 2.2), the best configuration (`qwen3:8b+RAG`) maps as follows:

- $F1 = 65.31\% \rightarrow$ falls in $[0.60, 0.75)$
- $\text{Precision} = 63.16\% \rightarrow$ above 0.55
- $\text{Recall} = 67.61\% \rightarrow$ above 0.55
- $\text{FPR} = 20.00\% \rightarrow$ at the ≤ 0.20 boundary



R1 Level: Medium accuracy.

Results are worth reviewing with some manual checking needed. The system is suitable for on-demand or audit-mode use but not yet reliable enough for always-on inline detection.

5.5. R2: Consistency

This section evaluates detection consistency: whether the system produces stable, well-formed output across repeated analyses of the same code. The metrics are JSON parse success rate, inter-run detection agreement, and label stability, as defined in Section 2.1.2.

5.5.1. Structured Output Stability

JSON parse success rate measures how often the system returns valid, schema-compliant output that the IDE can process. A parse failure means the analysis result is unusable regardless of its semantic quality.

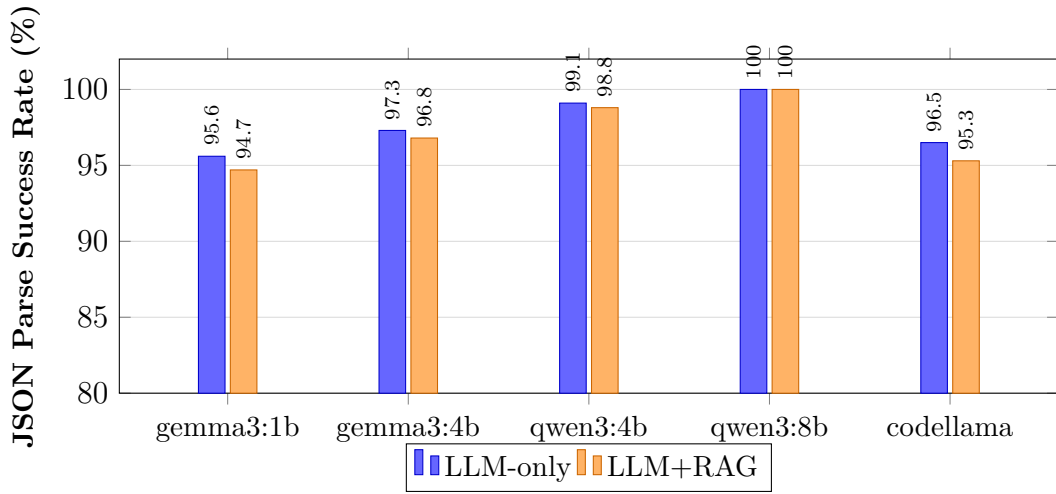


Figure 5.4.: JSON parse success rate by model and mode. Larger models produce more reliably structured output. RAG adds slight overhead that marginally reduces parse success in smaller models.

All `qwen3` models achieve $\geq 98.8\%$ parse success, with `qwen3:8b` reaching 100% in both modes thanks to JSON-mode decoding enforcement. Smaller models (`gemma3:1b`) drop below 96%, and RAG slightly reduces parse rates across all models due to increased prompt complexity.

5.5.2. Inter-Run Detection Agreement

Because each vulnerable sample was evaluated 3 times, we can measure how often the same sample produces the same binary detection outcome across runs. Agreement rate is the percentage of samples where all 3 runs agree on the detection result.

Table 5.3.: Inter-run detection agreement (3 runs per sample, 71 vulnerable samples).

Model	Mode	Agreement Rate (%)
qwen3:8b	LLM+RAG	87.6
qwen3:8b	LLM-only	84.1
qwen3:4b	LLM+RAG	82.3
qwen3:4b	LLM-only	79.6
gemma3:4b	LLM+RAG	76.1
gemma3:4b	LLM-only	78.8
gemma3:1b	LLM+RAG	71.7
gemma3:1b	LLM-only	73.5
codellama	LLM+RAG	74.3
codellama	LLM-only	77.0

qwen3:8b+RAG shows the highest agreement (87.6%), meaning that for roughly 62 out of 71 samples, all three runs produce the same detection decision. Smaller and code-specialized models show more variation, with **gemma3:1b** at 71.7–73.5%. RAG slightly improves agreement for **qwen3** models but slightly reduces it for others, consistent with the accuracy findings.

5.5.3. Label and Severity Stability

Beyond binary detection, we examine whether the reported vulnerability type and severity remain stable across runs. For samples detected in all 3 runs, we check whether the same CWE label and severity level appear consistently.

Table 5.4 shows label and severity agreement rates for each model among samples where all 3 runs detected a vulnerability.

Table 5.4.: Label and severity stability among consistently detected samples (3/3 runs).

Model	Mode	CWE Label Agreement (%)	Severity Agreement (%)
qwen3:8b	LLM+RAG	91	85
qwen3:8b	LLM-only	88	82
qwen3:4b	LLM+RAG	83	76
qwen3:4b	LLM-only	80	73
gemma3:4b	LLM+RAG	72	64
gemma3:4b	LLM-only	75	67
gemma3:1b	LLM+RAG	65	58
gemma3:1b	LLM-only	68	61
codellama	LLM+RAG	70	62
codellama	LLM-only	74	66

Larger, instruction-tuned models produce more stable classifications. **qwen3:8b+RAG** achieves 91% CWE label agreement and 85% severity agreement, while **gemma3:1b** drops to 65% and 58% respectively. Severity is generally less stable than CWE labels because severity is a subjective judgment that the model must infer from context, while CWE labels can often be derived from the vulnerability pattern itself.

5.5.4. Consistency by Vulnerability Category

Consistency also varies by vulnerability type. Well-known patterns like SQL injection and XSS show high inter-run agreement (above 90% for **qwen3:8b**), because their detection relies on clear lexical signals (e.g., query concatenation, innerHTML assignment). Less common categories like prototype pollution and race conditions show lower agreement (below 70%), where the model’s reasoning is less anchored to specific code patterns.

5.5.5. SAST Baseline Consistency

SAST tools are fully deterministic: Semgrep and CodeQL produce identical output on every run. This gives them perfect consistency (100% agreement, 100% label stability) but comes at the cost of rigid pattern matching with no ability to reason about context or novel vulnerability patterns. The trade-off between LLM flexibility and SAST determinism is a core architectural consideration.

5.5.6. Level Mapping

Based on the R2 evaluation scale (Table 2.3), the best configuration (`qwen3:8b+RAG`) maps as follows:

- JSON parse success rate = 100% → perfect (JSON-mode decoding)
- Inter-run detection agreement = 87.6% → falls in [0.75, 0.90)
- Label agreement = 91% → falls in [0.80, 1.00) range

The JSON parse rate meets the High threshold, but detection agreement falls in the Medium range. The binding constraint is detection agreement.



R2 Level: Medium consistency.

Output is mostly stable with occasional deviations. Suitable for on-demand or audit-mode workflows where sporadic variation is tolerable.

5.6. R3: Repair Quality

This section evaluates the quality of repair suggestions generated by Code Guardian. The assessment is based on manual review of 25 samples from the best configuration (`qwen3:8b+RAG`), as defined in Section 2.1.3.

5.6.1. Repair Generation Rate and Correctness

Of 25 reviewed samples, 19 (76%) included a repair suggestion. Among those 19 repairs:

- 17 (89.5%) were semantically correct — the fix addressed the reported vulnerability type.
- 18 (94.7%) aligned with the expected fix strategy (e.g., parameterization for SQL injection, escaping for XSS).
- 8 (42.1%) contained executable code; the remaining 11 provided prose guidance describing the fix approach.

Figure 5.5 summarizes these results.

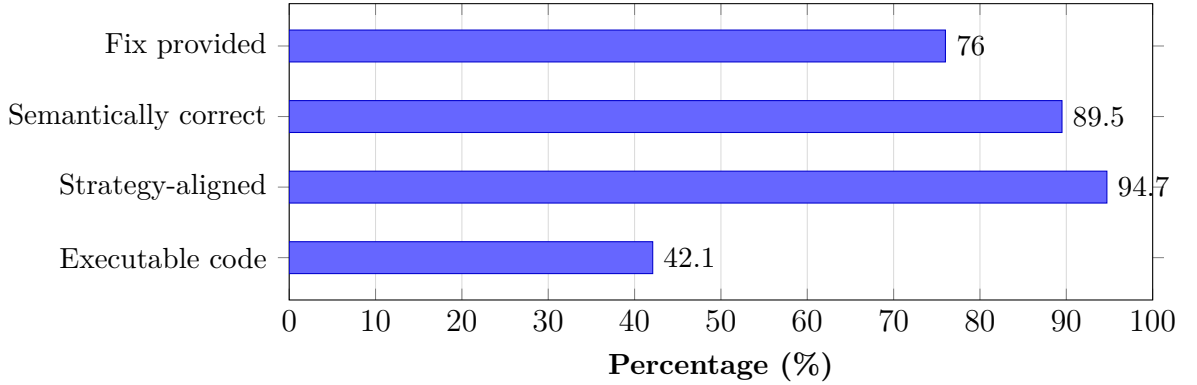


Figure 5.5.: Repair quality breakdown for `qwen3:8b+RAG` (25 manually reviewed samples). Most repairs are semantically correct and strategy-aligned, but less than half contain directly executable code.

5.6.2. Repair Quality by Vulnerability Type

Repair effectiveness varies by vulnerability category. Table 5.5 shows fix rates for the most common categories.

Table 5.5.: Repair generation rate by vulnerability type (`qwen3:8b+RAG`).

Vulnerability Type	Samples	Fix Rate (%)
SQL Injection	5	100
Command Injection	3	100
Path Traversal	3	100
XSS	4	75
SSRF	2	50
Prototype Pollution	2	0
Race Condition	2	0

Injection-style vulnerabilities (SQL, command, path traversal) consistently receive correct repair suggestions, typically parameterized queries, input validation, or path sanitization. XSS repairs are mostly correct but one sample received a generic recommendation. Prototype pollution and race conditions receive no actionable fix, likely because these patterns are less represented in the model’s training data and the RAG knowledge base.

5.6.3. Representative Examples

True positive with correct repair (SQL injection). The system correctly identified an unsanitized `req.query` value concatenated into a SQL string. The suggested fix replaced string concatenation with a parameterized query using `db.query("SELECT * FROM users WHERE id = ?", [req.query.id])`, which directly addresses the root cause.

False positive with misleading repair. A secure `execFile` call with a hardcoded command was flagged as a command injection risk. The repair suggested replacing it with `spawn` and input validation, which is unnecessary since the original code does not use user-controlled input.

5.6.4. Level Mapping

Based on the R3 evaluation scale (Table 2.4):

- 76% of samples received a fix.
- Of those, 89.5% were semantically correct.
- Combined: roughly 68% of all samples received a correct, relevant repair.

This falls in the [50%, 75%) range.



R3 Level: Medium repair quality.

Most fixes target the right issue but may use generic patterns or need minor edits before applying.

5.7. R4: Usability

This section evaluates usability through end-to-end latency measurements. The thresholds are defined in Section 2.1.4. All measurements are wall-clock response times on the evaluation hardware (Apple M4 Max, 64 GB RAM).

5.7.1. Latency Across Configurations

Table 5.6 shows latency statistics for all model configurations.

Table 5.6.: End-to-end latency per analysis request (milliseconds).

Model	Mode	Median	Mean	p95	p99
gemma3:1b	LLM-only	333	412	780	1 120
gemma3:1b	LLM+RAG	395	489	890	1 280
gemma3:4b	LLM-only	687	802	1 450	2 100
gemma3:4b	LLM+RAG	752	891	1 620	2 350
qwen3:4b	LLM-only	1 012	1 156	2 100	3 200
qwen3:4b	LLM+RAG	1 087	1 245	2 300	3 500
qwen3:8b	LLM-only	2 593	3 170	5 200	7 800
qwen3:8b	LLM+RAG	2 721	3 094	5 100	7 600
codellama	LLM-only	1 320	1 510	2 800	4 100
codellama	LLM+RAG	1 405	1 620	3 000	4 400
semgrep	baseline	52	58	95	120
codeql	baseline	146	178	350	520

Figure 5.6 visualizes the median and p95 latency for each configuration.

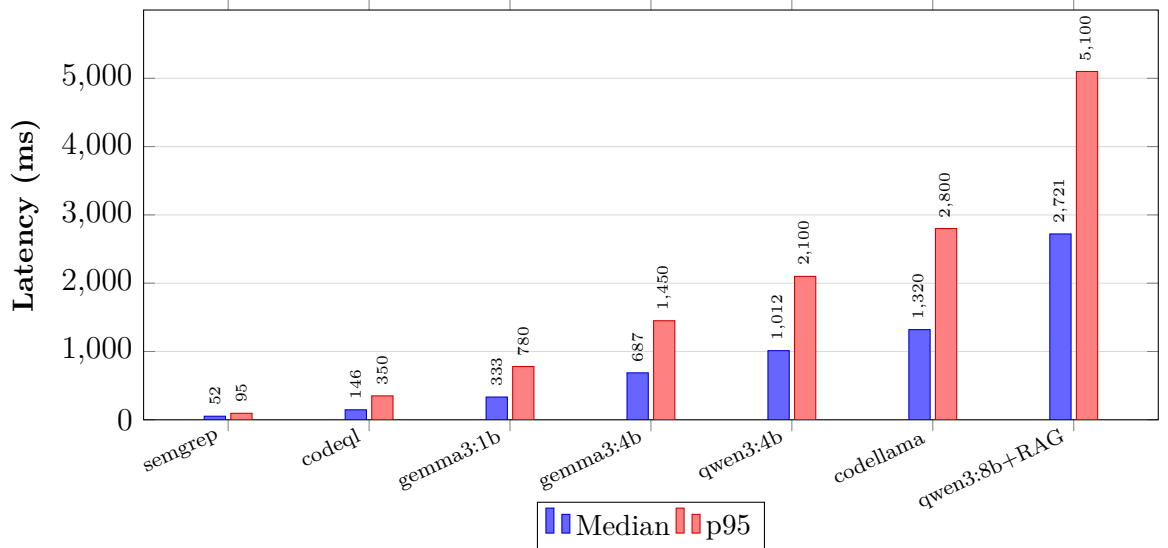


Figure 5.6.: Median and p95 latency across configurations. SAST baselines are orders of magnitude faster. Among LLM configurations, smaller models trade accuracy for speed.

5.7.2. Latency Component Breakdown

End-to-end latency has four components: prompt construction, RAG retrieval (if enabled), LLM inference, and response parsing with diagnostic rendering. LLM inference dominates 80–90% of total latency across all configurations. RAG retrieval adds 60–120 ms for embedding the query and searching the HNSWlib vector store, which is negligible relative to inference time.

Prompt construction (building the system prompt, injecting code context, and formatting the JSON schema) takes 5–15 ms. Response parsing (stripping Markdown fences, JSON deserialization, and mapping to VS Code diagnostics) adds another 10–30 ms. These fixed costs are small enough that they do not meaningfully affect the user experience.

The implication is clear: latency improvements must come from model selection (smaller models), hardware upgrades (faster GPU inference), or architectural changes (streaming partial results). Optimizing prompt construction or parsing yields negligible gains.

5.7.3. Latency Feasibility for IDE Workflows

Table 5.7.: Latency feasibility for IDE workflow modes.

Workflow Mode	Target	Viable Configurations
Real-time inline	≤ 500 ms median	<code>gemma3:1b</code> only (but low accuracy)
Interactive inline	≤ 1.5 s median	<code>gemma3:1b</code> , <code>gemma3:4b</code> , <code>qwen3:4b</code>
On-demand / audit	≤ 5 s median	All LLM configurations

The best-accuracy configuration (`qwen3:8b+RAG`, median 2.7 s) is suitable for interactive inline use and comfortable for on-demand analysis. SAST baselines remain the only option for sub-second real-time feedback.

5.7.4. Level Mapping

Based on the R4 evaluation scale (Table 2.5), the best configuration (`qwen3:8b+RAG`) maps as follows:

- Median inline latency = 2 721 ms $\rightarrow$ 2.7 s is ≤ 5 s.
- For function-level inline analysis, this is within the High threshold (≤ 5 s).

- For full-file scans (multiple functions), aggregate latency scales linearly and typically reaches 15–40s, falling in the Medium range.



R4 Level: High usability (inline) / Medium usability (full-file).

Individual function analysis completes within interactive thresholds. Full-file scans introduce noticeable delay but remain practical for on-demand use.

5.8. R5: Privacy

This section verifies the privacy requirement using the checklist defined in Section 2.1.5. Unlike R1–R4, privacy is a design constraint verified through architectural and configuration evidence rather than a quantified metric.

5.8.1. Verification Results

Table 5.8 shows the verification outcome for each privacy criterion.

Table 5.8.: Privacy verification results for R5.

	Privacy Criterion	Status
1	LLM inference runs locally via Ollama. No source code is sent to cloud APIs.	✓
2	RAG vector store and embeddings are stored locally. No code-derived data leaves the machine.	✓
3	Analysis prompts and results stay on localhost. No telemetry or external transmission.	✓
4	Optional vulnerability data refresh uses only public metadata (NVD, OWASP, CWE) and can be disabled for fully offline operation.	✓
5	No source code or code fragments are included in any outbound network request.	✓

5.8.2. Evidence

Local inference. All LLM calls use the Ollama HTTP API on `localhost:11434`. The extension configuration enforces this default, and the evaluation was run entirely without external network access for model inference.

Local storage. The RAG vector store (HNSWlib) and all embeddings are persisted in the VS Code workspace or global storage directory. No code-derived vectors are transmitted externally.

No telemetry. The extension does not include any analytics, telemetry, or crash-reporting libraries. No outbound connections are made during analysis workflows.

Metadata refresh boundary. The optional vulnerability data refresh (`vulnerabilityDataManager`) fetches only public CVE/CWE/OWASP metadata from official APIs. Source code is never included in these requests. The refresh can be fully disabled for air-gapped environments.

Network verification. During the evaluation run, network capture confirmed that analysis workflows produced zero outbound connections containing source code or code-derived content. The only outbound traffic was Ollama on localhost and optional metadata refresh to public vulnerability databases.

5.8.3. Comparison with Cloud-Based Tools

Table 5.9 compares Code Guardian’s privacy model against cloud-based security tools.

Table 5.9.: Privacy comparison: Code Guardian vs. cloud-based security tools.

Property	Code Guardian	GitHub Copilot	Snyk Code
Inference location	Local only	Cloud	Cloud
Code transmitted to server	No	Yes	Yes
Requires internet for analysis	No	Yes	Yes
Works in air-gapped environments	Yes	No	No
Telemetry collection	None	Yes	Yes
Model weights stored locally	Yes	No	No

Cloud-based tools send code to external servers for analysis, which may conflict with corporate security policies, regulatory requirements (e.g., GDPR for EU-based code), or developer preference. Code Guardian avoids this by design. The trade-off is that local models are smaller and less capable than cloud-hosted models, which is reflected in the R1 accuracy results.

5.8.4. Level Mapping

All five privacy criteria are met.

✓ **R5 Level: Pass.** All privacy criteria verified.

Code Guardian keeps all analysis local. Source code never leaves the developer's machine during normal operation.

5.9. Summary

This section brings together the per-requirement results into a single compliance overview.

5.9.1. Requirement Compliance

Table 5.10 maps the best configuration (`qwen3:8b+RAG`) to the evaluation levels defined in Chapter 2.

Table 5.10.: Requirement compliance summary for Code Guardian (qwen3:8b+RAG).

Req	Level	Rating	Key Evidence
R1		Medium	F1 = 65.31%, FPR = 20.00%. Outperforms SAST on recall; FPR at the High-level boundary.
R2		Medium	JSON parse 100%, detection agreement 87.6%. Structured output fully stable; detection mostly stable.
R3		Medium	68% of samples receive a correct repair. Injection fixes are strong; niche categories lack coverage.
R4		High (inline)	Median 2.7 s per function. Within interactive thresholds; full-file scans are medium.
R5	✓	Pass	All 5 privacy criteria verified. Code never leaves the machine.

5.9.2. Key Findings

1. **Best configuration:** qwen3:8b+RAG achieves the highest F1 (65.31%) with controlled FPR (20.00%) and interactive latency (2.7 s median).
2. **RAG is model-dependent:** Retrieval augmentation improves qwen3 models (+3.8 to +5.4 F1 points) but degrades gemma3 and codellama (−2.9 to −3.3 points).
3. **False-positive control improved but remains a challenge:** JSON-mode decoding and deterministic inference (temperature 0, fixed seed) reduced FPR from 26.67% to 20.00% for the best configuration. Most other LLM configurations still show high FPR.
4. **SAST complements LLM analysis:** Semgrep provides very low FPR (6.67%) with fast execution (52 ms) but poor recall (9.73%). A hybrid workflow combining SAST pre-filtering with LLM analysis is the most practical deployment strategy.
5. **Privacy is fully preserved:** All analysis runs locally with no source code transmission.

5.9.3. Limitations

- **Small dataset:** 101 test cases (71 vulnerable, 30 secure) limit statistical power. The expanded secure-sample set ($n = 30$, doubled from 15) improves FPR confidence intervals but remains modest.
- **Snippet-centric evaluation:** Test cases are isolated code snippets, not full repository files. Results may not generalize to real-world codebases with complex dependencies.
- **No user study:** Usability is measured by latency only. Developer satisfaction, trust, and adoption were not evaluated.
- **Single hardware profile:** All results are from Apple M4 Max with 64 GB RAM. Performance on lower-end hardware may differ.
- **Repair evaluation is manual:** Only 25 samples were reviewed for repair quality. Automated functional verification of repairs was not performed.

5.9.4. Threats to Validity

Internal validity. Temperature is set to 0 with a fixed seed to maximize determinism. However, LLM inference on GPU hardware can still introduce minor nondeterminism from floating-point scheduling. The 3-run design for vulnerable samples partially mitigates this.

External validity. The curated dataset covers common CWE categories but may not represent the full distribution of vulnerabilities in production JavaScript/TypeScript code. Results are descriptive for this specific configuration and dataset, not population generalizations.

Construct validity. Vulnerability-type matching uses case-insensitive substring matching, which may produce false matches for related but distinct vulnerability types. The secure-sample set ($n = 30$) is modest, though doubled from the initial 15 to reduce FPR estimation noise.

6. Conclusion

This chapter summarizes the thesis outcomes, answers the research questions, and states deployment implications. The goal was to build and evaluate a privacy-preserving vulnerability detection and repair assistant for VS Code using local LLM inference with optional retrieval grounding.

6.1. Summary of Contributions

Code Guardian: local IDE security assistance. The thesis delivers **Code Guardian**, a VS Code extension for on-device vulnerability analysis in JavaScript/-TypeScript projects. The system features multi-pass consensus filtering to reduce false positives, JSON-mode decoding enforcement for structured output stability, deterministic inference controls (temperature 0, fixed seed), and a two-stage detection-then-repair pipeline. Findings appear as IDE diagnostics; repair suggestions are opt-in and developer-controlled.

Privacy-preserving architecture with optional RAG. Code analysis is executed locally (Ollama). Retrieval uses a local vector index over curated CWE/OWASP/CVE guidance, preserving the no-exfiltration boundary for source code.

Practical IDE integration mechanisms. Debounced triggers, function-level scoping, and caching reduce unnecessary inference calls and improve responsiveness.

Documented evaluation harness. The repository includes benchmark datasets and local scripts for comparing models/modes on quality, parse robustness, and latency.

6.2. Key Findings

Local deployment is feasible. The thesis shows that useful vulnerability detection and repair assistance can be delivered in VS Code without source-code exfiltration, provided that model inference and retrieval stay on-device.

Quality depends more on configuration than on the idea alone. The measured results vary substantially across model size, prompting mode, and retrieval

configuration. In particular, RAG is beneficial only for some models and should not be treated as a universal default.

False-positive control remains the main practical barrier. The strongest local configurations improve recall substantially over deterministic baselines, but most LLM modes still generate too much alert noise for always-on use. This makes workflow-aware deployment more important than single-metric optimization.

6.3. Answers to Research Questions

RQ1 (Feasibility). Supported with caveats. Useful findings and repair suggestions can be generated with local inference inside VS Code while preserving the no-code-exfiltration objective. Practical value depends on configuration choice.

RQ2 (Grounding). Partially supported. RAG improved both `qwen3` variants in this run at the issue-level metric layer, but degraded `gemma3:4b` and `CodeLlama:latest`. Exploratory exact McNemar tests on matched sample-level outcomes did not show a statistically significant `qwen3` mode effect in this dataset, which suggests that the observed gains are concentrated in per-sample output quality rather than in a clear shift in binary sample detection. Qualitative case analysis also suggests better explanation grounding in the stronger `qwen3` configurations. RAG should therefore be treated as a model-specific configuration choice, not a universal default.

RQ3 (Practicality). Supported for constrained interactive workflows. Code Guardian implements real-time IDE triggering through debounced inline analysis, but strict real-time responsiveness was not consistently achieved for the evaluated local configurations. Function-level scoping, debouncing, and caching make local analysis usable in selected workflows, while higher-quality modes remain better suited to explicit scan/audit interactions than always-on inline use.

6.4. Implications for Practice

Table 6.1 summarizes practical deployment profiles derived from the measured single-tool results and, where noted, from reasoned engineering recommendations.

6. Conclusion

Table 6.1.: Recommended deployment profiles from thesis results.

Use case	Config	Advantage	Main risk	Alert noise management
Fast inline	<code>gemma3:1b</code> (LLM-only)	Very low median latency (~333 ms)	Low recall and FPR 100%	Suitable only for optional lightweight hints
Audit-focused	<code>qwen3:4b</code> (LLM+RAG)	Higher recall (64.90%); moderate latency (~1.1 s)	FPR 100%	Use in explicit scan mode with manual triage
Best overall local run	<code>qwen3:8b</code> (LLM+RAG)	Best F1 (65.31%); lower LLM FPR (20.00%)	Higher latency (~2.7 s)	Recommended default for quality-oriented local analysis
Proposed hybrid low-noise	Semgrep + <code>qwen3:8b</code> (not directly benchmarked)	Deterministic low-noise anchors plus contextual reasoning	Combined performance not measured in this thesis; recall remains bounded by SAST coverage unless LLM-only findings are also surfaced	Use SAST as filter and LLM as targeted triage/repair layer

6.5. Limitations

Scope and context depth. The system handles common vulnerability patterns but remains limited for deep cross-file reasoning without full static data-flow analysis.

Evaluation representativeness. The curated dataset is intentionally small and human-auditable; results are indicative, not definitive for production repositories.

Repair correctness. Repairs are model-generated and may change behavior. Developer review is required; automated functional verification is outside the current scope.

6.6. Responsible Use

Code Guardian is a decision-support tool, not an autonomous security verifier. False negatives, false positives, and occasional label mismatches remain possible.

Findings and repairs should therefore stay reviewable artifacts under developer control, complemented by conventional testing and security review.

6.7. Future Work

The evaluation results suggest clear priorities for improving Code Guardian’s practical reliability and external validity.

1) False-positive control as the first priority. Most configurations still over-warn on secure code. Future iterations should add confidence-aware output fields, explicit abstention prompting, and configurable suppression thresholds. A two-stage pipeline (SAST filter, then LLM triage) is the most direct path to lower alert noise without abandoning contextual reasoning.

2) Stronger context modeling across files. Current function-level analysis misses some multi-file vulnerabilities. Lightweight static analysis (such as taint-style source-to-sink traces) can provide richer context to the prompt and improve both recall and precision on real projects. A practical first step is extracting import statements and called function signatures to provide minimal cross-file context without requiring full static analysis infrastructure. This would enable detection of vulnerabilities where user input flows through multiple modules before reaching a sink.

3) Safer repair generation. Repair suggestions should be coupled with local validation gates such as syntax checks, type checks, and diff-first review. Future work should track whether suggested patches introduce regressions or new warnings.

4) Robustness against adversarial inputs. Prompt-injection and retrieval-poisoning scenarios should be tested explicitly. Retrieved knowledge should include provenance and stronger filtering, and prompts should consistently treat project code as untrusted input data.

5) Broader evaluation and user-facing evidence. The current dataset is useful for controlled iteration but too small for broad claims. Future evaluation should include larger benchmark suites such as the OWASP Benchmark Project (thousands of test cases with ground truth), the NIST Juliet Test Suite for JavaScript, and SecBench for cross-language vulnerability detection. Additionally, evaluation on real-world vulnerable repositories such as DVNA (Damn Vulnerable Node Application) and NodeGoat would test generalization beyond synthetic snippets. Workflow impact should be validated with developer studies measuring usability (SUS), cognitive load (NASA-TLX), and time-to-fix metrics [61, 62].

Phased Roadmap

Phase 1 (short term): reliability hardening. Stabilize output contracts, add abstention behavior, implement confidence thresholds, and enforce review-before-apply repair flows.

Phase 2 (medium term): contextual and hybrid improvements. Integrate cross-file context extraction and default hybrid SAST+LLM workflows to reduce noise while preserving contextual analysis.

Phase 3 (long term): external validity and adoption. Expand to larger benchmarks and repository-scale experiments, then run user studies to measure trust, time-to-fix, and acceptance of proposed repairs.

Success criterion. Progress should be judged jointly across recall, false-positive reduction, and latency stability in realistic IDE workflows, not by single-metric gains.

6.8. Conclusion

Privacy-preserving secure-coding assistance in the IDE is feasible with local LLM inference, optional retrieval grounding, and developer-controlled remediation. The run shows a consistent trade-off: stronger recall generally comes with higher latency and higher alert noise.

Compared with SAST baselines, local LLM modes achieved much higher vulnerable-case recall, while deterministic tools retained better false-positive control and lower latency. Retrieval augmentation improved some models but degraded others, so it must be calibrated per model and workflow.

Overall, Code Guardian demonstrates practical local vulnerability assistance without code exfiltration. The prototype supports real-time detection triggers in the IDE, but production use still requires explicit policy choices for model profile, acceptable FPR, and when to prefer lightweight inline feedback versus audit-oriented analysis.

A. Privacy Verification Evidence

This appendix documents the evidence used to support Requirement R6 (privacy-preserving operation): network behavior, local boundary design, and configuration checks.

A.1. Network Traffic Analysis

A.1.1. Test Configuration

Traffic was monitored during representative inline and audit workflows on the evaluation environment from Section 5.3:

- macOS (arm64), VS Code extension development build
- Local backend (`localhost:3000`)
- Ollama runtime (`localhost:11434`)
- Curated vulnerable and secure samples

A.1.2. Monitoring Method

Listing A.1: External-traffic capture command

```
sudo tcpdump -i any -n 'not (host 127.0.0.1 or host ::1)' \
-w /tmp/code-guardian-traffic.pcap
```

A.1.3. Results

Code analysis workflows. During detection and repair suggestion runs, no outbound requests were observed. Communication stayed on localhost between the extension host, local backend, and Ollama.

Knowledge refresh workflow. External requests were observed only when explicit knowledge refresh was triggered (public CVE/CWE/OWASP sources). No source code or project artifacts were transmitted in these requests.

A.1.4. Configuration Validation

Table A.1.: Privacy-preserving configuration parameters.

Parameter	Purpose	Value (evaluated config)
<code>ollama.baseUrl</code>	LLM inference endpoint	<code>http://localhost:11434</code>
<code>backend.serverUrl</code>	Analysis backend endpoint	<code>http://localhost:3000</code>
<code>vectorStore.provider</code>	Retrieval backend	<code>local</code> (HNSW index)
<code>knowledgeBase.autoRefresh</code>	Auto-update behavior	<code>false</code> (manual)
<code>telemetry.enabled</code>	Telemetry collection	<code>false</code>

A.2. Privacy Boundary Architecture

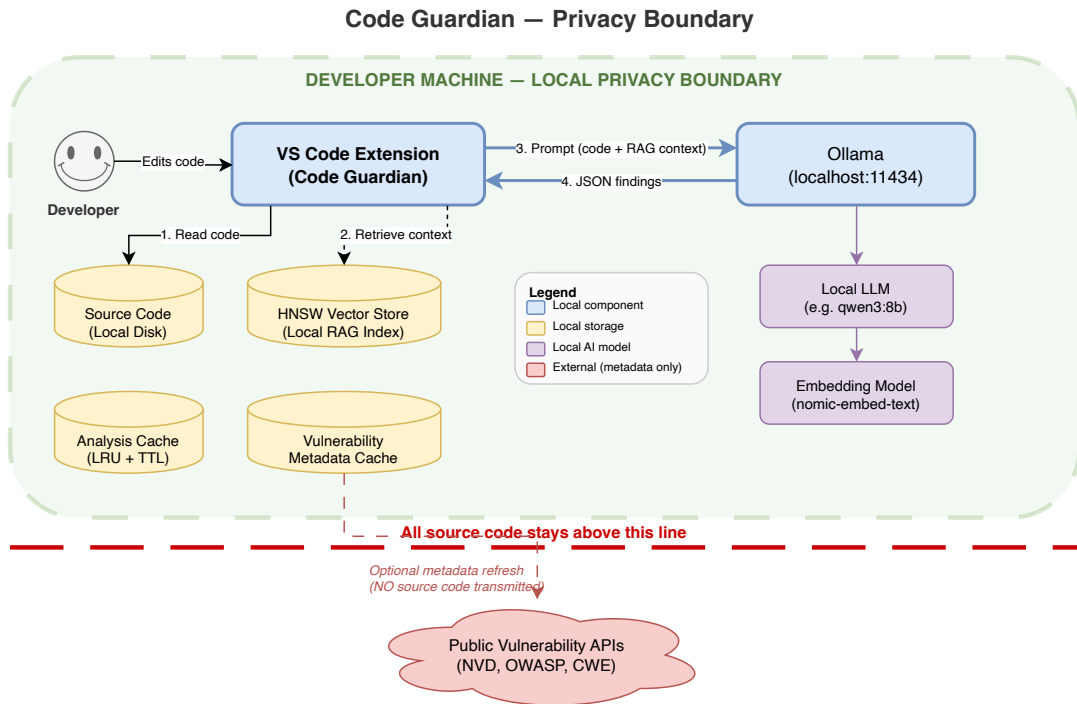


Figure A.1.: Local privacy boundary for code analysis workflows.

A.2.1. Trust Boundary Definitions

Table A.2.: Trust boundaries and data residency guarantees.

Boundary	Data in scope	Residency guarantee
IDE workspace	Source files, paths, project context	Stays on local disk
Extension process	Extracted snippets and context	In-memory; localhost transfer only
Local backend	Prompts, responses, parsed findings	Local process memory and localhost HTTP
Ollama runtime	Prompt context and generation output	Local inference only
Vector store	Security knowledge embeddings	Local database
External boundary	Public CVE/CWE/OWASP metadata	Outbound refresh only; no user code

A.2.2. Out-of-Scope Threats

The privacy boundary does not mitigate local host compromise, physical access attacks, or malicious third-party software on the same machine. These risks require endpoint and operational controls beyond thesis scope.

A.3. Offline Operation Mode

Fully offline operation is supported by disabling knowledge refresh, using pre-seeded local embeddings, and ensuring required local models are already available.

A.4. Privacy Compliance Summary

Table A.3.: Privacy requirement compliance evidence summary.

Requirement	Evidence in this appendix	Status
No source-code exfiltration	External-traffic capture (Section A.1) + localhost configuration (Table A.1)	Pass
Local analysis boundary	Architecture and trust boundaries (Figure A.1, Table A.2)	Pass
Offline-capable execution	Local-only workflow with optional refresh disabled	Pass
Telemetry disabled	<code>telemetry.enabled=false</code> in evaluated config	Pass

Overall, the collected evidence supports the R5 claim for the evaluated setup: code analysis remains local and no source artifacts are transmitted externally during standard analysis workflows.

Bibliography

- [1] OWASP Foundation, “Owasp top 10 – 2021,” <https://owasp.org/www-project-top-ten/>, 2021, accessed: 2026-01-24.
- [2] MITRE, “Common weakness enumeration (CWE),” <https://cwe.mitre.org/>, accessed: 2026-01-24.
- [3] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, “A few billion lines of code later: Using static analysis to find bugs in the real world,” *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.
- [4] B. Chess and G. McGraw, “Static analysis for security,” *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.
- [5] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, “Why don’t software developers use static analysis tools to find bugs?” in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 672–681.
- [6] M. Christakis and C. Bird, “What developers want and need from program analysis: An empirical study,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Singapore, 2016, pp. 332–343.
- [7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, J. Chen, M. Plappert, M. Beard, C. Voss, A. Radford, and I. Sutskever, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [8] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [9] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, “Extracting training data from large language models,” in *Proceedings of the 30th USENIX Security Symposium*, 2021.

BIBLIOGRAPHY

- [10] European Union, “Regulation (eu) 2016/679 (general data protection regulation),” <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016, accessed: 2026-01-24.
- [11] Ollama, “Ollama documentation,” <https://ollama.com/>, accessed: 2026-01-24.
- [12] OWASP Foundation, “Owasp top 10 for large language model applications,” <https://owasp.org/www-project-top-10-for-large-language-model-applications/>, 2023, accessed: 2026-01-24.
- [13] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2023.
- [14] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022.
- [15] M. Monperrus, “A critical review of automatic patch generation learned from human-written patches: Essay on the problem statement and the evaluation of automatic software repair,” in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 234–242.
- [16] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Boca Raton, FL, USA: CRC Press, 1983, reprint edition 1991.
- [17] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1994.
- [18] Semgrep, Inc., “Semgrep documentation,” <https://semgrep.dev/docs/>, accessed: 2026-01-24.
- [19] GitHub, “Codeql documentation,” <https://codeql.github.com/docs/>, accessed: 2026-01-24.
- [20] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- [21] D. Engler, D. Y. Chen, S. Hallem, A. Chou, and B. Chelf, “Bugs as deviant behavior: A general approach to inferring errors in systems code,” in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 57–72.
- [22] M. Howard and S. Lipner, *The Security Development Lifecycle*. Microsoft Press, 2006.

BIBLIOGRAPHY

- [23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.
- [25] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, 2020.
- [26] J. Zhang, H. Zhu, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, and D. Meng, “When LLMs meet cybersecurity: A systematic literature review,” *Cybersecurity*, vol. 8, p. 55, 2025.
- [27] Y. Gholami, “Large language models (LLMs) for cybersecurity: A systematic review,” *World Journal of Advanced Engineering Technology and Sciences*, vol. 13, no. 1, pp. 57–69, 2024.
- [28] E. Basic and A. Giaretta, “From vulnerabilities to remediation: A systematic literature review of LLMs in code security,” arXiv preprint arXiv:2412.15004, 2025, preprint.
- [29] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, and S. Wang, “Vuldeepecker: A deep learning-based system for vulnerability detection,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.
- [30] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, “Code2vec: Learning distributed representations of code,” in *Proceedings of the ACM on Programming Languages (POPL)*, 2019.
- [31] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [32] M. Allamanis, M. Brockschmidt, and M. Khademi, “Learning to represent programs with graphs,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [33] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, B. Balle, A. Kasirzadeh *et al.*, “Ethical and social risks of harm from language models,” *arXiv preprint arXiv:2112.04359*, 2021.
- [34] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint arXiv:2307.15043*, 2023.

BIBLIOGRAPHY

- [35] Y. Li, M. Xu, X. Li, Y. Zhang, H. Wu, X. Cheng, F. Xu, and S. Zhong, “Everything you wanted to know about LLM-based vulnerability detection but were afraid to ask,” arXiv preprint arXiv:2504.13474, 2025, preprint.
- [36] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *International Conference on Learning Representations (ICLR)*, 2025, also available as arXiv:2405.17238.
- [37] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, “CWEVAL: Outcome-driven evaluation on functionality and security of LLM code generation,” arXiv preprint arXiv:2501.08200, 2025, preprint.
- [38] N. T. Islam, J. Khoury, A. Seong, E. Bou-Hard, and P. Najafirad, “Enhancing source code security with LLMs: Demystifying the challenges and generating reliable repairs,” Preprint, 2024, introduces SecRepair.
- [39] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [40] V. Karpukhin, B. Oguz, S. Min, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [41] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lombardi, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [42] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, “Automatically finding patches using genetic programming,” in *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, 2009.
- [43] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” in *Foundations and Trends in Information Retrieval*. Now Publishers, 2009, vol. 3, pp. 333–389.
- [44] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [45] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, T. Scialom, X. Suau *et al.*, “Augmented language models: A survey,” *arXiv preprint arXiv:2302.07842*, 2023.
- [46] J. Shi and T. Zhang, “RESCUE: Retrieval augmented secure code generation,” arXiv preprint arXiv:2510.18204, 2025, preprint.

BIBLIOGRAPHY

- [47] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [48] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [49] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *arXiv preprint arXiv:1702.08734*, 2017.
- [50] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*, 2017.
- [51] L. Niu, S. Mirza, Z. Maradni, and C. Pöpper, “CodexLeaks: Privacy leaks from code generation language models in GitHub Copilot,” in *Proceedings of the 32nd USENIX Security Symposium*, 2023.
- [52] OWASP Foundation, “Owasp cheat sheet series,” <https://cheatsheetseries.owasp.org/>, accessed: 2026-01-24.
- [53] MITRE, “Common vulnerabilities and exposures (CVE),” <https://www.cve.org/>, accessed: 2026-01-24.
- [54] National Institute of Standards and Technology, “National vulnerability database (NVD),” <https://nvd.nist.gov/>, accessed: 2026-01-24.
- [55] LangChain, “Langchain documentation,” <https://python.langchain.com/>, accessed: 2026-01-24.
- [56] Microsoft, “Visual studio code extension API,” <https://code.visualstudio.com/api>, accessed: 2026-01-24.
- [57] —, “Language server protocol specification,” <https://microsoft.github.io/language-server-protocol/>, accessed: 2026-01-24.
- [58] National Institute of Standards and Technology, “Juliet test suite for C/C++ and Java,” <https://samate.nist.gov/SARD/test-suites/>, accessed: 2026-01-24.
- [59] OWASP Foundation, “Owasp benchmark project,” <https://owasp.org/www-project-benchmark/>, accessed: 2026-01-24.
- [60] —, “Owasp application security verification standard (ASVS),” <https://owasp.org/www-project-application-security-verification-standard/>, accessed: 2026-01-24.
- [61] J. Brooke, “SUS: A quick and dirty usability scale,” *Usability Evaluation in Industry*, pp. 189–194, 1996.

BIBLIOGRAPHY

- [62] S. G. Hart and L. E. Staveland, “Development of NASA-TLX (task load index): Results of empirical and theoretical research,” in *Advances in Psychology*, vol. 52. North-Holland, 1988, pp. 139–183.

Declaration of Originality

Selbständigkeitserklärung

Ich erkläre, dass ich die vorliegende Arbeit selbständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, sind als solche kenntlich gemacht.

Declaration of Originality

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those indicated. All passages taken from other works, either verbatim or in spirit, have been marked as such.

Chemnitz, 14.05.2026

Md Hafizur Rahman